

Lógica Computacional — Resumo Teórico

Gonçalo Sousa

Atualizado: Semana 2 (Aulas 1–4)

Conteúdo

Aula 1 — Introdução à Lógica e Sintaxe da Lógica Proposicional	3
O que é a lógica e porque interessa à informática	3
Proposições	3
Conectivos lógicos	4
Linguagem da lógica proposicional	4
Semântica: valorações e tabelas de verdade	9
Satisfazibilidade, tautologias e contradições	10
Aula 2 — Consequência Semântica, Equivalência e Conectivos Completos	11
A relação de satisfação $\models_v$	11
Satisfazibilidade de um conjunto de fórmulas	12
Consequência semântica e equivalência	12
Propriedades da relação $\models$	15
Quantas proposições podem ser simultaneamente verdadeiras	19
Conjuntos completos de conectivos	19
Aplicação: modelar problemas em lógica proposicional	20
Aula 3 — Formas Normais e Satisfazibilidade	24
Literais e forma normal negativa (FNN)	24
Forma normal disjuntiva (FND)	25
Satisfazibilidade de uma FND	27
Forma normal conjuntiva (FNC)	28
Fórmulas de Horn	31
Aula 4 — Satisfazibilidade: cláusulas e o algoritmo de Davis-Putnam	35
O problema SAT	35
Cláusulas	35
Propagação unitária	36
O algoritmo DLL	37

Aula 1 — Introdução à Lógica e Sintaxe da Lógica Proposicional

O que é a lógica e porque interessa à informática

A **lógica** estuda os métodos e princípios que distinguem um raciocínio correto de um incorreto. A lógica formal nasce com Aristóteles (silogismos: regras de inferência como o *modus ponens* — de $A \rightarrow B$ e A infere-se B), mas só se torna **lógica matemática** a partir do séc. XIX (Boole, Frege, Hilbert, Gödel...), associada a problemas de fundamentação da matemática: teoria de modelos, teoria da dedução (prova) e sistemas axiomáticos.

Atenção

Cuidado com silogismos “parecidos” que não são válidos. “Todos os homens são mortais; Sócrates é homem; logo Sócrates é mortal” é válido. Mas “Todos os homens são mortais; Sócrates era mortal; logo todos os homens são Sócrates” **não é válido**, apesar de ter a mesma aparência superficial — a forma do argumento é que importa, não o facto de as frases “soarem” parecidas.

Uma **lógica** (no sentido abstrato) é composta por três partes:

1. uma **linguagem/sintaxe** — um alfabeto e regras para construir *fórmulas*;
2. uma **semântica** — uma noção do que é verdadeiro (pode haver várias interpretações semânticas para a mesma sintaxe);
3. um **sistema de dedução** — um conjunto finito de regras que permite obter fórmulas novas a partir de outras (raciocinar). Interessa que seja **correto** (*sound*): tudo o que se deduz é verdadeiro; e idealmente **completo**: tudo o que é verdadeiro é dedutível (nem sempre é possível ter as duas coisas — é isso que o curso vai explorar com os teoremas de correção e completude).

Neste curso trabalha-se sobretudo com a **lógica clássica** (proposicional e de primeira ordem). Há outras famílias de lógica — lógica de ordem superior, lógicas sub-estruturais (ex: intuicionista, onde $\neg\neg A \not\leftrightarrow A$, ao contrário do que veremos aqui) e lógicas modais (onde a verdade é parametrizada, ex: por tempo — “Hoje é segunda-feira” só é verdade nalguns “mundos”). É útil ter estas categorias em mente para perceber porque é que a lógica clássica que vamos estudar faz as suposições que faz (por exemplo, todas as proposições são verdadeiras ou falsas — nunca as duas nem nenhuma — o chamado *princípio do terceiro excluído*, que não vale em lógica intuicionista).

A lógica é relevante para a informática em várias frentes: circuitos digitais (lógica booleana), inteligência artificial, autómatos finitos (correspondem a fórmulas de lógica monádica de segunda ordem), XML/documentos estruturados, classes de complexidade caracterizadas por classes de fórmulas, linguagens de programação (programação lógica como Prolog; sistemas de tipos = sistemas dedutivos), verificação e especificação formal (hardware, software, protocolos de segurança), dedução automática (SAT/SMT solvers) e provadores de teoremas assistidos (Coq, Isabelle, Agda).

Proposições

Definição — Proposição

Uma **proposição** é uma frase declarativa à qual se pode atribuir um valor de verdade, **V** (verdadeiro) ou **F** (falso) — e apenas um dos dois. Frases que não afirmam nada com um valor de verdade definido (perguntas, ordens, expressões sem verbo, ou frases com variáveis livres cujo valor de verdade depende de algo não especificado) **não são proposições**.

Exemplo resolvido — Exercício 1.1 (lab, proplogic.pdf) — 5 das 8 alíneas, escolhidas para não repetir a mesma ideia

Para cada frase, indicar se é proposição e, sendo, o seu valor de verdade (as 8 alíneas do enunciado testam só 5 ideias distintas — ver nota a seguir à caixa):

(a) “O Porto é a capital de Portugal.” — É proposição. **F** (a capital é Lisboa). (*caso base: frase factual simples, com valor de verdade definido.*)

(d) “ $1 + 2 + 3$ ” — **Não é proposição**: é uma expressão/termo (um número), não uma afirmação. Não diz nada que possa ser verdadeiro ou falso.

(c) “Hei de comparar-te a um dia de verão?” — **Não é proposição**: é uma pergunta (interrogativa), não uma afirmação — não tem valor de verdade (nota que isto é um motivo *diferente* do de (d): ali era por não ser uma afirmação de todo, aqui é por ser uma pergunta).

(g) “ $x + y$ é um número par.” — **Não é proposição**: x e y são *variáveis livres* — o valor de verdade depende de que valores lhes atribuímos (é verdade para $x = 1, y = 1$, falso para $x = 1, y = 2$). A frase só passa a ter um valor de verdade fixo depois de fixarmos x e y .

(h) “Existe um x tal que $7 + x$ é um número par.” — É proposição, e **V**: apesar de ter uma variável, o quantificador “existe um x ” **fecha** a variável — a frase inteira já não depende de nenhum x externo (basta $x = 1$: $7 + 1 = 8$ é par). Compara com (g): a diferença entre uma variável livre (não é proposição) e uma variável quantificada (é proposição) só fica completamente formal em lógica de primeira ordem, mais à frente no curso — mas já vale a pena notar a distinção agora.

Atenção

As alíneas (b), (e) e (f) do Exercício 1.1 ficam de fora de propósito — testam exatamente a mesma ideia de (a) (frase factual simples, decidir V/F), só com outro assunto. Não precisas de as fazer; se quiseres confirmar que percebeste, é literalmente o mesmo passo de (a) com números diferentes.

Conectivos lógicos

Cada proposição simples é representada por uma **variável proposicional** ($p, q, r, \dots$), e proposições compostas obtêm-se combinando-as com **conectivos**:

Conectivo	Símbolo	Aridade	Outros símbolos equivalentes
Conjunção (“e”)	$\wedge$	2	$\&, \&\&, \cdot$
Disjunção (“ou”)	$\vee$	2	$, +$
Negação (“não”)	$\neg$	1	$\sim, -, !$
Implicação (“se...então”)	$\rightarrow$	2	$\Rightarrow, \supset$

Linguagem da lógica proposicional

Definição — Alfabeto

A linguagem da lógica proposicional é construída a partir dos símbolos primitivos:

- um conjunto **numerável** de variáveis proposicionais $\mathcal{V}_{Prop} = \{p, q, r, \dots, p_1, \dots\}$;
- os conectivos lógicos $\wedge, \vee, \neg, \rightarrow$;
- os parêntesis (e).

Definição — Fórmula bem formada

O conjunto das **fórmulas** $(\phi, \psi, \theta, \dots)$ é definido indutivamente:

1. uma variável proposicional p é uma fórmula;
2. se ϕ é fórmula, então $(\neg\phi)$ é fórmula;
3. se ϕ e ψ são fórmulas, então $(\phi \wedge \psi)$, $(\phi \vee \psi)$ e $(\phi \rightarrow \psi)$ são fórmulas.

Nada mais é fórmula. Exemplo: $((p \wedge (\neg p)) \rightarrow (\neg(p \wedge (q \vee r))))$.

Esta definição indutiva é o que permite desenhar a **árvore sintática** de uma fórmula: a raiz é o conectivo principal, e os seus filhos são as fórmulas que ele combina (recursivamente, até chegar às variáveis, que são as folhas).

Convenção para omitir parêntesis

Escrever todos os parêntesis torna as fórmulas ilegíveis, por isso usa-se uma convenção de precedência (tal como em aritmética, onde $\times$ tem prioridade sobre $+$):

- os parêntesis mais exteriores podem ser omitidos;
- $\neg$ tem prioridade sobre $\wedge$;
- $\wedge$ tem prioridade sobre $\vee$;
- $\vee$ tem prioridade sobre $\rightarrow$;
- $\wedge$ e $\vee$ são **associativos à esquerda**;
- $\rightarrow$ é **associativo à direita**.

Exemplo resolvido

- $\phi \wedge \psi \vee \theta$ abrevia $((\phi \wedge \psi) \vee \theta)$ (o $\wedge$ “agarra” primeiro).
- $p \wedge \neg p \rightarrow \neg(p \wedge (q \vee r))$ corresponde a $((p \wedge (\neg p)) \rightarrow (\neg(p \wedge (q \vee r))))$.
- $p \rightarrow q \rightarrow r$ corresponde a $p \rightarrow (q \rightarrow r)$ (associatividade à direita do $\rightarrow$: lê-se “de dentro para fora”, começando pelo lado direito).

Definição — Subfórmula

Uma **subfórmula imediata** de ψ é: (1) nenhuma, se ψ é uma variável; (2) ϕ , se $\psi = \neg\phi$; (3) ϕ e ψ' , se $\psi = \phi \wedge \psi'$, $\phi \vee \psi'$ ou $\phi \rightarrow \psi'$.

ϕ é **subfórmula** de ψ se ϕ é subfórmula imediata de ψ , ou se existe θ tal que ϕ é subfórmula de θ e θ é subfórmula de ψ (fecho transitivo das subfórmulas imediatas). Por convenção, ψ é sempre subfórmula de si própria.

O exercício seguinte junta a convenção de precedência, a árvore sintática e a noção de subfórmula — a melhor forma de treinar os três em conjunto é desenhar a árvore completa e depois ler as subfórmulas diretamente dos nós dessa árvore (cada nó interno, incluindo a raiz, é uma subfórmula; as folhas são as variáveis).

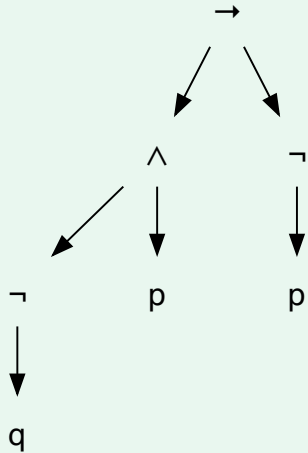
Exemplo resolvido — Exercício 1.3(a) (lab, proplogic.pdf)

Fórmula: $p \wedge \neg q \rightarrow \neg p$.

Passo 1 — reintroduzir os parêntesis pela convenção de precedência. $\neg$ agarra primeiro: $\neg q$ e $\neg p$. Depois $\wedge$: $p \wedge (\neg q)$. Por fim $\rightarrow$ (o conectivo de menor prioridade, por isso é a raiz):

$$(p \wedge (\neg q)) \rightarrow (\neg p)$$

Passo 2 — árvore sintática (raiz = conectivo principal, $\rightarrow$):



Passo 3 — lista de subfórmulas (lidas diretamente dos nós da árvore, da raiz para as folhas):
 $p \wedge \neg q \rightarrow \neg p$ (a própria fórmula), $p \wedge \neg q$, $\neg p$, p , $\neg q$, q .

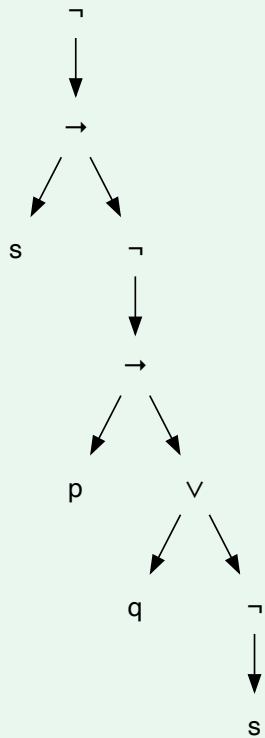
Exemplo resolvido — Exercício 1.3(c) (lab, proplogic.pdf)

Fórmula: $\neg(s \rightarrow (\neg(p \rightarrow (q \vee \neg s))))$.

Aqui os parêntesis já vêm quase todos explícitos no enunciado — só falta decompor “de fora para dentro” para identificar cada conectivo principal. Vamos dar um nome a cada subfórmula à medida que aparece, de dentro para fora (é a ordem inversa da leitura, mas é a ordem em que a árvore se constrói):

- $A = q \vee \neg s$ (disjunção de q com $\neg s$)
- $B = p \rightarrow A$
- $C = \neg B$
- $D = s \rightarrow C$
- $E = \neg D$ (a fórmula toda)

Árvore sintática:



Subfórmulas: E (a própria fórmula), $D = s \rightarrow \neg(p \rightarrow (q \vee \neg s))$, s , $C = \neg(p \rightarrow (q \vee \neg s))$, $B = p \rightarrow (q \vee \neg s)$, p , $A = q \vee \neg s$, q , $\neg s$, s (repara que s aparece **duas vezes** na árvore, como dois nós diferentes — mas é a mesma fórmula, só se lista uma vez no conjunto de subfórmulas).

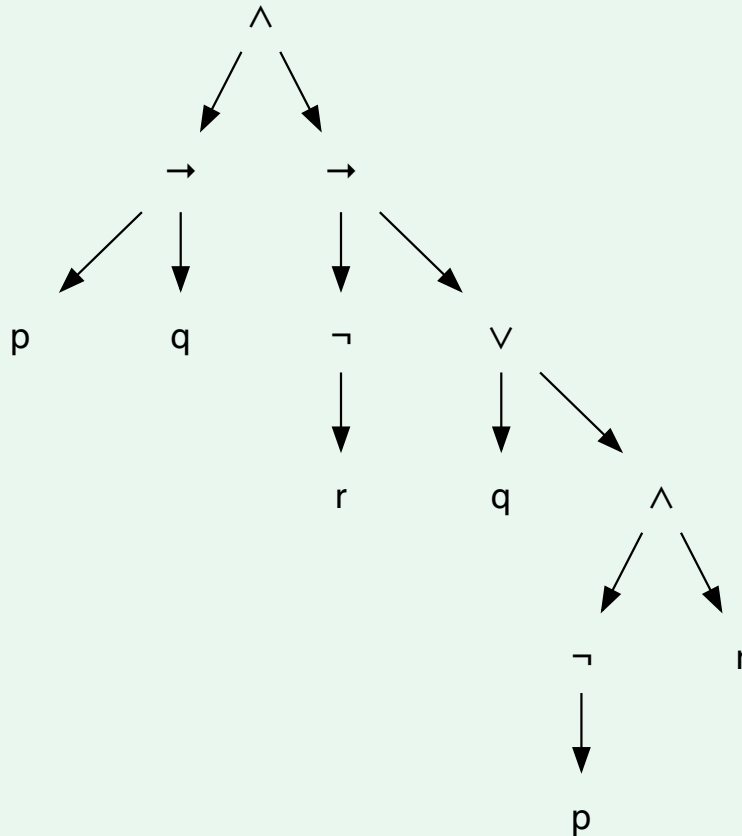
Exemplo resolvido — Exercício 1.3(e) (lab, proplogic.pdf)

Fórmula: $(p \rightarrow q) \wedge (\neg r \rightarrow (q \vee (\neg p \wedge r)))$.

De novo os parêntesis já resolvem a ambiguidade; só falta identificar a estrutura. O conectivo principal é o $\wedge$ mais exterior, que combina dois ramos:

- ramo esquerdo: $L = p \rightarrow q$
- ramo direito: $M = \neg r \rightarrow M_2$, onde $M_2 = q \vee M_1$ e $M_1 = \neg p \wedge r$

Árvore sintática:



Subfórmulas: a fórmula toda, $L = p \rightarrow q$, p , q , $M = \neg r \rightarrow (q \vee (\neg p \wedge r))$, $\neg r$, r , $M_2 = q \vee (\neg p \wedge r)$, q , $M_1 = \neg p \wedge r$, $\neg p$, p , r .

Atenção

Nota adicional (não estava explícito nos slides, mas é essencial para não errar nos exercícios): quando reconstróis os parêntesis pela convenção de precedência, faz sempre isso **antes** de tentar desenhar a árvore — tentar fazer as duas coisas ao mesmo tempo é a fonte mais comum de erros. Também repara que o conectivo **principal** de uma fórmula (a raiz da árvore) é sempre o de **menor prioridade** entre os que aparecem “soltos” (não dentro de parêntesis explícitos) — por isso em 1.3(a) a raiz é $\rightarrow$ e não $\wedge$, mesmo aparecendo $\wedge$ mais à esquerda na leitura da fórmula.

Prática agora — Linguagem e árvores sintáticas (proplogic.pdf)

- **1.2** — traduzir cada fórmula para português. É mecânico: lê cada conectivo com a tabela da secção “Conectivos lógicos” ($\rightarrow$ = “se... então”). Atenção à (d) vs. (e) vs. (f): são três implicações diferentes, não digas o mesmo por palavras diferentes.
- **1.3 (b) e (d)** — árvore sintática e subfórmulas, mesma técnica de (a)/(c)/(e) acima. Na (b) compara com a (a): os parêntesis mudam o conectivo principal.

Soluções, uma alínea de cada vez: *SOLUCOES_Logica.html*

Semântica: valorações e tabelas de verdade

Definição — Valoração (atribuição de valores de verdade)

Uma **valoração** é uma função $v : \mathcal{V}_{Prop} \rightarrow \{V, F\}$ que atribui um valor de verdade a cada variável proposicional. Estende-se ao conjunto das fórmulas por indução na estrutura:

1. $v(p)$ já está definido, para $p \in \mathcal{V}_{Prop}$;
2. $v(\neg\phi) = V$ se $v(\phi) = F$, e $v(\neg\phi) = F$ se $v(\phi) = V$;
3. $v(\phi \wedge \psi) = V$ sse $v(\phi) = V$ e $v(\psi) = V$ (senão F);
4. $v(\phi \vee \psi) = V$ sse $v(\phi) = V$ ou $v(\psi) = V$ (senão F);
5. $v(\phi \rightarrow \psi) = F$ sse $v(\phi) = V$ e $v(\psi) = F$ (senão V).

Atenção

O caso mais contra-intuitivo é a implicação: $\phi \rightarrow \psi$ só é **falsa** quando o antecedente é verdadeiro e o consequente é falso. Em particular, sempre que o antecedente ϕ é **falso**, a implicação inteira é **verdadeira**, independentemente do consequente (“de uma premissa falsa pode concluir-se o que quiser” — *ex falso quodlibet*). Isto vai ser usado constantemente nas provas “sem construir a tabela de verdade” (Exercício 1.14, Aula 2).

ϕ	$\neg\phi$	ϕ	ψ	$\phi \wedge \psi$	ϕ	ψ	$\phi \vee \psi$	ϕ	ψ	$\phi \rightarrow \psi$
F	V	F	F	F	F	F	F	F	F	V
V	F	F	V	F	F	V	V	F	V	V
		V	F	F	V	F	V	V	F	F
		V	V	V	V	V	V	V	V	V

Uma fórmula ϕ com n variáveis proposicionais tem 2^n valorações possíveis (cada variável pode ser V ou F , independentemente das outras) — constrói-se uma **tabela de verdade** com uma linha por valoração, calculando o valor de ϕ coluna a coluna (dos operadores mais “internos” para o mais “externo”, exatamente como na árvore sintática).

Exemplo resolvido — Exercício 1.4(b) (lab, proplogic.pdf)

Construir a tabela de verdade de $\neg q \wedge (p \rightarrow q)$ (2 variáveis, logo 4 linhas):

p	q	$\neg q$	$p \rightarrow q$	$\neg q \wedge (p \rightarrow q)$
V	V	F	V	F
V	F	V	F	F
F	V	F	V	F
F	F	V	V	V

A fórmula só é verdadeira quando p e q são ambos falsos. Isto não é coincidência: por equivalências semânticas (ver mais abaixo), $\neg q \wedge (p \rightarrow q) \Leftrightarrow \neg q \wedge (\neg p \vee q) \Leftrightarrow (\neg q \wedge \neg p) \vee (\neg q \wedge q) \Leftrightarrow (\neg p \wedge \neg q) \vee F \Leftrightarrow \neg p \wedge \neg q$ — exatamente o que a tabela mostra.

Exemplo resolvido — Exercício 1.4(c) (lab, proplogic.pdf)

Construir a tabela de verdade de $(p \wedge q) \rightarrow \neg(p \vee q)$:

p	q	$p \wedge q$	$p \vee q$	$\neg(p \vee q)$	$(p \wedge q) \rightarrow \neg(p \vee q)$
V	V	V	V	F	F
V	F	F	V	F	V
F	V	F	V	F	V
F	F	F	F	V	V

Só é falsa na linha $p = V, q = V$: aí o antecedente $p \wedge q$ é verdadeiro mas o consequente $\neg(p \vee q)$ é falso (porque $p \vee q$ é verdadeiro). Em todas as outras linhas o antecedente $p \wedge q$ já é falso, o que torna a implicação automaticamente verdadeira (ver nota acima sobre a implicação).

Pratica agora — Tabelas de verdade (proplogic.pdf)

- **1.4 (a) e (d)** — tabelas de verdade, mesma técnica de (b)/(c) acima. Na (d), antes de acabar, repara em que linhas o antecedente é verdadeiro: é aí que tudo se decide.

Soluções, uma alínea de cada vez: SOLUCOES_Logica.html

Satisfazibilidade, tautologias e contradições**Definição — Satisfazibilidade, tautologia, contradição**

Uma fórmula ϕ é:

- **satisfazível** se existe uma valoração v tal que $v(\phi) = V$ — escreve-se $\models_v \phi$ e diz-se que v **satisfaz** ϕ ;
- **tautologia** se **todas** as valorações v dão $v(\phi) = V$ — escreve-se $\models \phi$ (ex: $\models p \vee \neg p$, o princípio do terceiro excluído);
- **contradição** se **todas** as valorações v dão $v(\phi) = F$ — escreve-se $\not\models \phi$ (ex: $\not\models p \wedge \neg p$).

Uma fórmula é **insatisfazível** sse é uma contradição. Uma fórmula satisfazível mas não tautologia diz-se por vezes **contingente**: é verdadeira nalgumas valorações e falsa noutras.

Aula 2 — Consequência Semântica, Equivalência e Conectivos Completos

A relação de satisfação $\models_v$

A extensão de v às fórmulas (aula anterior) pode ser reescrita como uma relação $\models_v$, definida indutivamente na estrutura da fórmula — é apenas outra notação para a mesma coisa, mas é a notação usada daqui em diante:

Definição — Relação de satisfação

Dada uma valoração v , a relação $\models_v$ define-se por:

1. $\models_v p$ sse $v(p) = V$;
2. $\models_v \neg\phi$ sse $\not\models_v \phi$;
3. $\models_v \phi \wedge \psi$ sse $\models_v \phi$ e $\models_v \psi$;
4. $\models_v \phi \vee \psi$ sse $\models_v \phi$ ou $\models_v \psi$;
5. $\models_v \phi \rightarrow \psi$ sse $\not\models_v \phi$ ou $\models_v \psi$.

Diz-se que v **satisfaz** ϕ se $\models_v \phi$.

Exemplo resolvido

Com $v(p) = V, v(q) = v(r) = F$, para saber se $\models_v (p \rightarrow (q \vee r)) \vee (r \rightarrow \neg p)$ basta que **um** dos dois lados da disjunção seja satisfeito. O lado direito, $r \rightarrow \neg p$, é satisfeito porque $\not\models_v r$ (implicação com antecedente falso). Logo $\models_v (p \rightarrow (q \vee r)) \vee (r \rightarrow \neg p)$ — nem foi preciso avaliar o lado esquerdo.

Exemplo resolvido — Exercício 1.5(b) (lab, proplogic.pdf)

Com $v(p) = V$ e $v(q) = F$, verificar se $\models_v (p \vee \neg q) \wedge (\neg p \wedge q)$.

O segundo conjunto, $\neg p \wedge q$: temos $v(p) = V$ logo $\not\models_v \neg p$ (já falha aqui). Como um conjunto ($\wedge$) precisa dos **dois** lados satisfeitos e um deles já falha, $\not\models_v \neg p \wedge q$. Logo, mesmo sem avaliar o primeiro conjunto, a conjunção inteira falha: $\not\models_v (p \vee \neg q) \wedge (\neg p \wedge q)$.

Exemplo resolvido — Exercício 1.5(e) (lab, proplogic.pdf)

Com $v(p) = V$ e $v(q) = F$, verificar se $\models_v ((p \rightarrow q) \rightarrow p) \rightarrow p$.

$p \rightarrow q$: $v(p) = V, v(q) = F$, logo $\not\models_v p \rightarrow q$ (antecedente verdadeiro, consequente falso — o único caso em que a implicação falha). $(p \rightarrow q) \rightarrow p$: antecedente falso ($\not\models_v p \rightarrow q$), logo esta implicação é automaticamente satisfeita: $\models_v (p \rightarrow q) \rightarrow p$. Fórmula toda: $((p \rightarrow q) \rightarrow p) \rightarrow p$: antecedente satisfeito ($\models_v (p \rightarrow q) \rightarrow p$) e consequente satisfeito ($\models_v p$, porque $v(p) = V$) — logo $\models_v ((p \rightarrow q) \rightarrow p) \rightarrow p$.

Frequente em exame

$((p \rightarrow q) \rightarrow p) \rightarrow p$ é a **Lei de Peirce**, e é de facto uma **tautologia** (é verdadeira para *qualquer* v , não só para o v deste exercício — podes confirmar construindo a tabela completa com as 4 valorações de p, q). É um resultado clássico e frequentemente pedido: é um dos exemplos-padrão de tautologia que **não é válida em lógica intuicionista** (relembra a nota da Aula 1 sobre lógicas sub-estruturais) — distingue-a da lógica clássica que estamos a estudar.

Atenção

Nota adicional — armadilha comum: **não confundir** $(p \rightarrow q) \rightarrow p$ (que aparece na Lei de Peirce acima) com $(q \rightarrow p) \rightarrow p$ (parecida, mas com p e q trocados no antecedente). Ao contrário da primeira, $(q \rightarrow p) \rightarrow p$ **não** é tautologia — falha, por exemplo, com $v(p) = F$: nesse caso $q \rightarrow p = \neg q$ (verdadeiro só se q for falso), e se $v(q) = F$ então $q \rightarrow p$ é verdadeiro, tornando $(q \rightarrow p) \rightarrow p$ igual a $V \rightarrow F = F$. A posição exata de cada variável na fórmula importa — nunca assumas que uma fórmula “parecida” com uma tautologia conhecida também o é.

Prática agora — A relação $\models_v$ (proplogic.pdf)

- **1.5 (a), (c) e (d)** — com $v(p) = V$, $v(q) = F$, avalia de dentro para fora, como em (b)/(e) acima. *Soluções, uma alínea de cada vez: SOLUCOES_Logica.html*

Satisfazibilidade de um conjunto de fórmulas**Definição — Satisfazibilidade de um conjunto**

Seja Γ um conjunto de fórmulas. Uma valoração v **satisfaz** Γ (notação $\models_v \Gamma$) se satisfaz **todas** as fórmulas de Γ : $\forall \psi \in \Gamma, \models_v \psi$. Γ é **satisfazível** se existe alguma valoração que o satisfaz.

Consequência semântica e equivalência**Definição — Consequência semântica ($\models$)**

ϕ é **consequência semântica** de Γ (notação $\Gamma \models \phi$) se **toda** valoração v que satisfaz Γ também satisfaz ϕ .

Casos especiais de notação:

- $\emptyset \models \phi$ escreve-se $\models \phi$ — equivale a ϕ ser tautologia (nenhuma condição sobre v : tem de valer sempre).
- Se $\Gamma = \{\psi\}$, escreve-se $\psi \models \phi$.
- Se $\psi \models \phi$ e $\phi \models \psi$, ψ e ϕ dizem-se **semanticamente equivalentes**, escreve-se $\psi \Leftrightarrow \phi$ (têm exatamente o mesmo valor de verdade em qualquer valoração).

Definição — Leis de equivalência semântica

Lei	Equivalência
Comutatividade	$\phi \wedge \psi \Leftrightarrow \psi \wedge \phi$; $\phi \vee \psi \Leftrightarrow \psi \vee \phi$
Lei de De Morgan	$\neg(\phi \wedge \psi) \Leftrightarrow \neg\phi \vee \neg\psi$; $\neg(\phi \vee \psi) \Leftrightarrow \neg\phi \wedge \neg\psi$
Associatividade	$(\phi \wedge \psi) \wedge \theta \Leftrightarrow \phi \wedge (\psi \wedge \theta)$; idem para $\vee$
Idempotência	$\phi \vee \phi \Leftrightarrow \phi$; $\phi \wedge \phi \Leftrightarrow \phi$
Distributividade	$(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta)$; $(\phi \vee \psi) \wedge \theta \Leftrightarrow (\phi \wedge \theta) \vee (\psi \wedge \theta)$
Dupla negação	$\neg\neg\phi \Leftrightarrow \phi$
Implicação	$\phi \rightarrow \psi \Leftrightarrow \neg\phi \vee \psi$

Estas leis funcionam como uma “álgebra booleana”: permitem transformar uma fórmula noutra equivalente sem ter de recorrer à tabela de verdade — substituindo subfórmulas pelo seu equivalente, passo a passo (tal como se faz álgebra normal, mas com $\wedge, \vee, \neg$ em vez de $\times, +$).

Atenção — Leis extra para simplificar

Nota adicional (não estão na tabela dos slides nem na do enunciado, mas são precisas para **simplificar** uma fórmula até se ver o que ela é — cada uma confirma-se com uma tabela de 2 linhas). V e F representam uma fórmula sempre verdadeira e uma sempre falsa:

Lei	Equivalência
Complementaridade	$\phi \vee \neg\phi \Leftrightarrow V$; $\phi \wedge \neg\phi \Leftrightarrow F$
Elemento neutro	$\phi \vee F \Leftrightarrow \phi$; $\phi \wedge V \Leftrightarrow \phi$
Elemento absorvente	$\phi \vee V \Leftrightarrow V$; $\phi \wedge F \Leftrightarrow F$
Absorção	$\phi \vee (\phi \wedge \psi) \Leftrightarrow \phi$; $\phi \wedge (\phi \vee \psi) \Leftrightarrow \phi$

Exemplo resolvido — Exercício 1.6(a) (lab, proplogic.pdf)

Mostrar que $\neg(p \rightarrow q) \Leftrightarrow p \wedge \neg q$, usando as leis (sem tabela de verdade):

$$\neg(p \rightarrow q) \stackrel{\text{(implicação)}}{\Leftrightarrow} \neg(\neg p \vee q) \stackrel{\text{(De Morgan)}}{\Leftrightarrow} \neg\neg p \wedge \neg q \stackrel{\text{(dupla negação)}}{\Leftrightarrow} p \wedge \neg q$$

Cada passo substitui uma subfórmula pelo seu equivalente segundo uma única lei da tabela — é assim que se lê/escreve qualquer prova por equivalências.

Exemplo resolvido — Exercício 1.6(e) (lab, proplogic.pdf)

Mostrar que $(p \vee q) \rightarrow r \Leftrightarrow (p \rightarrow r) \wedge (q \rightarrow r)$:

$$(p \vee q) \rightarrow r \stackrel{\text{(implicação)}}{\Leftrightarrow} \neg(p \vee q) \vee r \stackrel{\text{(De Morgan)}}{\Leftrightarrow} (\neg p \wedge \neg q) \vee r$$

Agora aplica-se a distributividade — $(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta)$ — com $\phi = \neg p$, $\psi = \neg q$, $\theta = r$:

$$(\neg p \wedge \neg q) \vee r \stackrel{\text{(distributividade)}}{\Leftrightarrow} (\neg p \vee r) \wedge (\neg q \vee r) \stackrel{\text{(implicação, 2x)}}{\Leftrightarrow} (p \rightarrow r) \wedge (q \rightarrow r)$$

Atenção

Guarda a equivalência do Exercício 1.6(e) — $(p \vee q) \rightarrow r \Leftrightarrow (p \rightarrow r) \wedge (q \rightarrow r)$ — porque serve para **confirmar** o Exercício 1.13(d) mais abaixo: como $(p \rightarrow r) \wedge (q \rightarrow r)$ implica $(p \rightarrow r) \vee (q \rightarrow r)$, a consequência tem de valer. Em exame, porém, a 1.13(d) prova-se com a definição de $\models_v$, como lá está feito.

Exemplo resolvido — Exercício 1.8(a) e 1.8(b) (lab, proplogic.pdf) — distribuir a implicação sobre a conjunção

(a) $(p \rightarrow q) \wedge (p \rightarrow r) \Leftrightarrow p \rightarrow (q \wedge r)$ — **é válida**. Prova pelas leis:

$$(p \rightarrow q) \wedge (p \rightarrow r) \Leftrightarrow (\neg p \vee q) \wedge (\neg p \vee r) \stackrel{\text{(distributividade)}}{\Leftrightarrow} \neg p \vee (q \wedge r) \Leftrightarrow p \rightarrow (q \wedge r)$$

(b) $(p \rightarrow r) \wedge (q \rightarrow r) \Leftrightarrow (p \wedge q) \rightarrow r$ — **NÃO é válida**. Contraexemplo: $v(p) = V, v(q) = F, v(r) = F$. Lado esquerdo: $p \rightarrow r = V \rightarrow F = F$; logo $(p \rightarrow r) \wedge (q \rightarrow r) = F$. Lado direito: $p \wedge q = V \wedge F = F$; logo $(p \wedge q) \rightarrow r = F \rightarrow F = V$. Lado esquerdo é F , lado direito é V — **não são equivalentes**.

Atenção

Compara com atenção (a) e (b): olham parecidas (ambas “distribuem $\rightarrow$ sobre $\wedge$ ”), mas só a que distribui $\wedge$ do lado do **consequente** (a) é válida — distribuir sobre o **antecedente** (b) já não é. É um erro muito fácil de cometer sem verificar — usa sempre as leis ou um contraexemplo para confirmar, nunca “por analogia visual”.

Classificar uma fórmula simplificando-a (Exercício 1.9)

As leis servem também para **classificar** uma fórmula (tautologia, contradição ou satisfazível), que é o que o Exercício 1.9 pede — e o enunciado diz o método: “*You can start by simplifying the formulas using semantic equivalences*”. Não se faz a tabela de verdade.

Definição — Método: classificar por equivalências

1. **Eliminar as implicações** ($\phi \rightarrow \psi \Leftrightarrow \neg\phi \vee \psi$) e **empurrar as negações** para dentro (De Morgan, dupla negação).
2. **Simplificar** com as leis (complementaridade, absorção, elemento neutro/absorvente) até a fórmula ficar numa forma que se lê logo.
3. **Ler a classificação** do resultado:
 - chegou a $V \rightarrow$ **tautologia**;
 - chegou a $F \rightarrow$ **contradição**;
 - chegou a outra coisa (ex.: $p \vee \neg q$) $\rightarrow$ **satisfazível** (nem tautologia nem contradição). O enunciado pede então **uma valoração que a torna verdadeira e outra que a torna falsa**: lêem-se da forma simplificada, e **confirmam-se na fórmula original**.

Exemplo resolvido — Exercício 1.9(d) (lab, proplogic.pdf) — tautologia

$$\begin{aligned}
 & p \rightarrow (q \rightarrow (p \vee q)) \\
 \Leftrightarrow & \neg p \vee (\neg q \vee (p \vee q)) && (\text{implicação, 2 vezes}) \\
 \Leftrightarrow & (\neg p \vee p) \vee (\neg q \vee q) && (\text{associatividade e comutatividade do } \vee) \\
 \Leftrightarrow & V \vee V && (\text{complementaridade, 2 vezes}) \\
 \Leftrightarrow & V && (\text{elemento absorvente})
 \end{aligned}$$

Chegou a V : é **tautologia**. (Com o $\vee$ tudo ao mesmo nível, os parêntesis podem reorganizar-se à vontade — foi isso que permitiu juntar $\neg p$ com p e $\neg q$ com q .)

Exemplo resolvido — Exercício 1.9(j) (lab, proplogic.pdf) — contradição

Truque: o segundo conjunto é a **negação** do primeiro.

$$\neg(p \vee \neg q) \xLeftrightarrow{\text{(De Morgan)}} \neg p \wedge \neg\neg q \xLeftrightarrow{\text{(dupla negação)}} \neg p \wedge q$$

Então, chamando $X = p \vee \neg q$:

$$(p \vee \neg q) \wedge (\neg p \wedge q) \Leftrightarrow X \wedge \neg X \xLeftrightarrow{\text{(complementaridade)}} F$$

Chegou a F : é **contradição**.

Exemplo resolvido — Exercício 1.9(i) (lab, proplogic.pdf) — satisfazível

$$\begin{aligned}
 & (p \vee q) \rightarrow (p \wedge q) \\
 \Leftrightarrow & \neg(p \vee q) \vee (p \wedge q) \quad (\text{implicação}) \\
 \Leftrightarrow & (\neg p \wedge \neg q) \vee (p \wedge q) \quad (\text{De Morgan})
 \end{aligned}$$

Já não simplifica mais: não é V nem F . Lê-se diretamente: é verdadeira quando p e q são ambos falsos ou ambos verdadeiros (é “ $p \leftrightarrow q$ ”). Logo é **satisfazível, mas não tautologia nem contradição**.

- **Verdadeira** com $p = V, q = V$ (o segundo termo, $p \wedge q$, é V). Confirmação na original: $(V \vee V) \rightarrow (V \wedge V) = V \rightarrow V = V$.
- **Falsa** com $p = V, q = F$ (os dois termos têm um literal falso). Confirmação na original: $(V \vee F) \rightarrow (V \wedge F) = V \rightarrow F = F$.

Prática agora — Consequência e equivalência (proplogic.pdf)

- **1.6 (b), (c) e (d)** — provar as equivalências, pela tabela ou pelas leis (tenta pelas leis: é o que treina para o exame). Na (b), usa $\varphi \rightarrow \psi \Leftrightarrow \neg\varphi \vee \psi$ dos dois lados.
- **1.8 (c)** — é válida ou não? Mesma comparação de (a)/(b) acima: verifica com as leis de distributividade ou encontra um contraexemplo.
- **1.9 (c)** — classificar **pelo método acima** (simplificar com as leis; nada de tabela). Se não for tautologia nem contradição, dá uma valoração que a torna verdadeira e outra que a torna falsa.
- **1.9 (h)** — o mesmo, com 3 variáveis. Pista: depois de eliminar as implicações, procura uma **absorção**.

As restantes alíneas da 1.9 ((a), (b), (e), (f), (g), (k)) são a mesma simplificação repetida — não precisas de as fazer todas.

Soluções, uma alínea de cada vez: [SOLUCOES_Logica.html](#)

Propriedades da relação $\models$

As afirmações seguintes envolvem $\Gamma \models \theta$ de forma mais abstrata (sem fixar fórmulas concretas) — testam se percebeste bem a **definição** de consequência semântica, não cálculo. A técnica geral: para mostrar que uma afirmação é **verdadeira**, usa diretamente a definição; para mostrar que é **falsa**, basta um contraexemplo concreto (escolher Γ, Σ, θ específicos).

Definição — Método: provar com a definição de $\models_v$ (formato de exame)

Nota adicional: é assim que a professora resolve, e é o formato aceite em exame. Explicar “por palavras” não chega.

Para mostrar $\Gamma \models \phi$ (ou $\models \phi$, com $\Gamma = \emptyset$):

1. “Seja v uma valoração tal que $\models_v \Gamma$.” (Para $\models \phi$: “seja v uma valoração qualquer.”)
2. Abrir cada fórmula pelo seu **conectivo principal**, com a definição de $\models_v$ (regras 1–5 acima). Quando a regra dá um “ou”, **separar em casos** e fechar cada um.
3. Chegar a $\models_v \phi$ em todos os casos. Concluir: “como v é arbitrária, $\Gamma \models \phi$.” ■

Alternativa: prova por absurdo. Supõe-se $\not\models_v \phi$ e abre-se com as regras da negação. Costuma ser a mais curta quando ϕ é uma implicação, porque $\not\models_v \phi \rightarrow \psi$ força $\models_v \phi$ e $\not\models_v \psi$. Se se chega a $\models_v \theta$ e $\not\models_v \theta$, essa v não existe.

Para mostrar que é falso: dá-se uma **valoração concreta** e verifica-se com as regras que $\models_v \gamma$ para cada $\gamma \in \Gamma$ e $\not\models_v \phi$.

Regras da negação, que saem das regras 2–5:

- $\not\models_v \phi \wedge \psi$ sse $\not\models_v \phi$ ou $\not\models_v \psi$
- $\not\models_v \phi \vee \psi$ sse $\not\models_v \phi$ e $\not\models_v \psi$
- $\not\models_v \phi \rightarrow \psi$ sse $\models_v \phi$ e $\not\models_v \psi$

Modus ponens pode ser citado: se $\models_v \phi$ e $\models_v \phi \rightarrow \psi$, então $\models_v \psi$ (pela regra 5, $\not\models_v \phi$ é impossível).

Exemplo resolvido — Exercício 1.16(a), (b) e (f) (lab, proplogic.pdf) — monotonicidade

(a) Se $\Gamma \models \theta$ e $\Gamma \subseteq \Sigma$, então $\Sigma \models \theta$ — VERDADEIRO.

Seja v uma valoração tal que $\models_v \Sigma$, isto é, $\models_v \psi$ para todo o $\psi \in \Sigma$. Seja $\psi \in \Gamma$. Como $\Gamma \subseteq \Sigma$, $\psi \in \Sigma$, logo $\models_v \psi$. Portanto $\models_v \Gamma$. Como $\Gamma \models \theta$, por definição de $\models$ tem-se $\models_v \theta$. Como v é arbitrária, $\Sigma \models \theta$. ■
(Intuição: **acrescentar premissas nunca destrói** uma consequência já válida. Chama-se **monotonicidade**.)

(b) Se $\Sigma \models \theta$ e $\Gamma \subseteq \Sigma$, então $\Gamma \models \theta$ — FALSO (é o recíproco de (a)).

Contraexemplo: $\Sigma = \{p, p \rightarrow q\}$, $\Gamma = \{p\}$, $\theta = q$. Tem-se $\Gamma \subseteq \Sigma$.

- $\Sigma \models q$: seja v tal que $\models_v p$ e $\models_v p \rightarrow q$. Como $\models_v p \rightarrow q$ sse $\not\models_v p$ ou $\models_v q$, e $\not\models_v p$ é impossível, tem-se $\models_v q$.
- $\Gamma \not\models q$: com $v(p) = V$, $v(q) = F$, tem-se $\models_v p$, isto é, $\models_v \Gamma$, mas $\not\models_v q$.

As hipóteses valem e a conclusão não. **Menos premissas podem já não bastar** para a mesma conclusão.

(f) Se $\Sigma \models \theta$ e $\Sigma \subseteq \Gamma$, então $\Sigma \cup \Gamma \models \theta$ — VERDADEIRO. Se $\Sigma \subseteq \Gamma$, então $\Sigma \cup \Gamma = \Gamma$. A afirmação passa a ser “se $\Sigma \models \theta$ e $\Sigma \subseteq \Gamma$, então $\Gamma \models \theta$ ”, que é a **alínea (a)** com os nomes Γ e Σ trocados. Já está provada. ■

Atenção

A lição de (a)/(b)/(f): $\Gamma \models \theta$ é **monótono ao acrescentar premissas** (mais hipóteses, no máximo, só ajudam a concluir mais coisas — nunca deixam de valer uma consequência já estabelecida), mas **não é monótono ao remover premissas** — retirar uma hipótese pode fazer uma conclusão válida deixar de o ser. Isto explica intuitivamente porque é que (a) e (f) são verdadeiros (só acrescentam) e (b) é falso (remove).

Exemplo resolvido — Exercício 1.13 (lab, proplogic.pdf)**(a) $\models (p \wedge \neg p) \rightarrow (p \wedge \neg r)$ — VERDADEIRO.**Seja v uma valoração qualquer.

$$\models_v (p \wedge \neg p) \rightarrow (p \wedge \neg r) \text{ sse } \not\models_v p \wedge \neg p \text{ ou } \models_v p \wedge \neg r$$

Ora, $\models_v p \wedge \neg p$ sse $\models_v p$ e $\models_v \neg p$, isto é, sse $\models_v p$ e $\not\models_v p$, o que é impossível. Logo $\not\models_v p \wedge \neg p$ e, portanto, $\models_v (p \wedge \neg p) \rightarrow (p \wedge \neg r)$. Como v é arbitrária, $\models (p \wedge \neg p) \rightarrow (p \wedge \neg r)$. ■

(b) $\models (q \vee \neg r) \rightarrow (q \wedge \neg q)$ — FALSO.

Procura-se v com $\models_v (q \vee \neg r) \rightarrow (q \wedge \neg q)$, isto é, com $\models_v q \vee \neg r$ e $\not\models_v q \wedge \neg q$. A segunda condição vale sempre, como em (a). Basta então $\models_v q \vee \neg r$.

Contraexemplo: $v(q) = V$, $v(r) = V$.

- $\models_v q$, logo $\models_v q \vee \neg r$.
- $\models_v q$, logo $\not\models_v \neg q$, logo $\not\models_v q \wedge \neg q$.
- Então $\not\models_v (q \vee \neg r) \rightarrow (q \wedge \neg q)$, e a fórmula **não** é tautologia.

(c) $\{(p \rightarrow q) \vee r, ((p \rightarrow q) \vee r) \rightarrow \neg r, (p \rightarrow q) \rightarrow (q \rightarrow r)\} \models \neg q$ — VERDADEIRO.Seja v tal que $\models_v (p \rightarrow q) \vee r$, $\models_v ((p \rightarrow q) \vee r) \rightarrow \neg r$ e $\models_v (p \rightarrow q) \rightarrow (q \rightarrow r)$.

1. $\models_v ((p \rightarrow q) \vee r) \rightarrow \neg r$ sse $\not\models_v (p \rightarrow q) \vee r$ ou $\models_v \neg r$. O primeiro é impossível (é a 1.^a premissa), logo $\models_v \neg r$, isto é, $\not\models_v r$.
2. $\models_v (p \rightarrow q) \vee r$ sse $\models_v p \rightarrow q$ ou $\models_v r$. Por 1, $\not\models_v r$, logo $\models_v p \rightarrow q$.
3. $\models_v (p \rightarrow q) \rightarrow (q \rightarrow r)$ sse $\not\models_v p \rightarrow q$ ou $\models_v q \rightarrow r$. Por 2, $\models_v p \rightarrow q$.
4. $\models_v q \rightarrow r$ sse $\not\models_v q$ ou $\models_v r$. Por 1, $\not\models_v r$, logo $\not\models_v q$, isto é, $\models_v \neg q$.

Como v é arbitrária, $\{(p \rightarrow q) \vee r, ((p \rightarrow q) \vee r) \rightarrow \neg r, (p \rightarrow q) \rightarrow (q \rightarrow r)\} \models \neg q$. ■**(d) $\{(p \vee q) \rightarrow r\} \models (p \rightarrow r) \vee (q \rightarrow r)$ — VERDADEIRO.**Seja v tal que $\models_v (p \vee q) \rightarrow r$. Queremos $\models_v (p \rightarrow r) \vee (q \rightarrow r)$.

$$\models_v (p \vee q) \rightarrow r \text{ sse } \not\models_v p \vee q \text{ ou } \models_v r$$

- **Caso 1:** $\models_v r$. Como $\models_v p \rightarrow r$ sse $\not\models_v p$ ou $\models_v r$, tem-se $\models_v p \rightarrow r$. Logo $\models_v (p \rightarrow r) \vee (q \rightarrow r)$.
- **Caso 2:** $\not\models_v p \vee q$, isto é, $\not\models_v p$ e $\not\models_v q$. Como $\not\models_v p$, tem-se $\models_v p \rightarrow r$. Logo $\models_v (p \rightarrow r) \vee (q \rightarrow r)$.

Em ambos os casos $\models_v (p \rightarrow r) \vee (q \rightarrow r)$. Como v é arbitrária, $\{(p \vee q) \rightarrow r\} \models (p \rightarrow r) \vee (q \rightarrow r)$. ■

Atenção: os dois casos estão ligados por um “ou”. Basta um deles acontecer. Não se pode juntar “ $v(r) = V$ e $v(p) = F$ e $v(q) = F$ ”.

Exemplo resolvido — Exercício 1.14(a) e 1.14(d) (lab, proplogic.pdf) — sem tabela de verdade**(a) $\models p \rightarrow (((q \rightarrow p) \rightarrow p) \rightarrow r)$ — FALSO.**Procura-se v com $\models_v$ da fórmula:

$$\begin{aligned} \models_v p \rightarrow (((q \rightarrow p) \rightarrow p) \rightarrow r) \text{ sse } \models_v p \text{ e } \models_v ((q \rightarrow p) \rightarrow p) \rightarrow r \\ \models_v ((q \rightarrow p) \rightarrow p) \rightarrow r \text{ sse } \models_v (q \rightarrow p) \rightarrow p \text{ e } \models_v r \end{aligned}$$

Com $\models_v p$, a condição $\models_v (q \rightarrow p) \rightarrow p$ vale sempre, porque o conseqüente é satisfeito. Basta então $v(p) = V$ e $v(r) = F$.**Contraexemplo:** $v(p) = V$, $v(q) = V$, $v(r) = F$.

- $\models_v p$, logo $\models_v (q \rightarrow p) \rightarrow p$ (conseqüente satisfeito).
- $\models_v (q \rightarrow p) \rightarrow p$ e $\models_v r$, logo $\models_v ((q \rightarrow p) \rightarrow p) \rightarrow r$.
- $\models_v p$ e $\models_v ((q \rightarrow p) \rightarrow p) \rightarrow r$, logo $\models_v p \rightarrow (((q \rightarrow p) \rightarrow p) \rightarrow r)$. Não é tautologia.

(d) $\models ((p \wedge q) \rightarrow (s \vee t)) \rightarrow ((p \rightarrow s) \vee (q \rightarrow t))$ — VERDADEIRO. Prova por absurdo.Suponha-se que existe v com $\models_v$ da fórmula. Então:

1. $\models_v (p \wedge q) \rightarrow (s \vee t)$ e $\models_v (p \rightarrow s) \vee (q \rightarrow t)$ (regra da negação de $\rightarrow$).
2. $\models_v (p \rightarrow s) \vee (q \rightarrow t)$ sse $\models_v p \rightarrow s$ e $\models_v q \rightarrow t$, isto é: $\models_v p$, $\models_v s$, $\models_v q$, $\models_v t$.
3. De $\models_v p$ e $\models_v q$: $\models_v p \wedge q$. De $\models_v s$ e $\models_v t$: $\models_v s \vee t$.
4. Então $\models_v (p \wedge q) \rightarrow (s \vee t)$, o que contradiz 1.

Logo não existe tal v : a fórmula é satisfeita por todas as valorações, $\models ((p \wedge q) \rightarrow (s \vee t)) \rightarrow ((p \rightarrow s) \vee (q \rightarrow t))$. ■**Atenção**

Nota adicional sobre “sem tabela de verdade”: nos dois casos começa-se por **supor $\models_v$ da fórmula** e abrir com as regras da negação. Se se chega a uma valoração consistente, como em (a), essa valoração é o contraexemplo, e verifica-se regra a regra. Se se chega a $\models_v \theta$ e $\not\models_v \theta$, como em (d), é uma prova por absurdo de que a fórmula é tautologia. (Na 1.9, pelo contrário, o enunciado manda simplificar pelas leis.)

Pratica agora — Propriedades de $\models$ (proplogic.pdf)

- **1.15** — mesma ideia do Exercício 1.16 acima, mas com $\models$ e $\Leftrightarrow$ entre fórmulas genéricas $\varphi, \psi, \theta, \gamma$: se for verdadeiro, prova com a definição de $\models_v$ (método acima); se for falso, instancia com fórmulas concretas ($\varphi = p$, ...) e dá a valoração do contraexemplo, verificada regra a regra.

Soluções, uma alínea de cada vez: SOLUCOES_Logica.html

Quantas proposições podem ser simultaneamente verdadeiras

Exemplo resolvido — Exercício 1.7 (lab, proplogic.pdf)

Atribuindo valores de verdade a p, q, r , quantas das cinco disjunções $p \vee \neg q$, $\neg p \vee q$, $q \vee r$, $q \vee \neg r$, $\neg q \vee \neg r$ podem ser simultaneamente verdadeiras?

Como há só 3 variáveis, é viável construir a tabela completa (8 linhas) e contar, em cada linha, quantas das 5 fórmulas são V:

p	q	r	$p \vee \neg q$	$\neg p \vee q$	$q \vee r$	$q \vee \neg r$	$\neg q \vee \neg r$	nº de V
V	V	V	V	V	V	V	F	4
V	V	F	V	V	V	V	V	5
V	F	V	V	F	V	F	V	3
V	F	F	V	F	F	V	V	3
F	V	V	F	V	V	V	F	3
F	V	F	F	V	V	V	V	4
F	F	V	V	V	V	F	V	4
F	F	F	V	V	F	V	V	4

O **máximo** é **5** (todas simultaneamente verdadeiras), atingido em $p = V, q = V, r = F$ — por exemplo, verifica-se que $\neg q \vee \neg r$ dá $F \vee V = V$ porque $r = F$. O **mínimo** ao longo de todas as valorações é **3** (nunca é possível ter menos de 3 verdadeiras ao mesmo tempo, e isso acontece em três linhas diferentes da tabela).

Conjuntos completos de conectivos

Definição — Conjunto completo de conectivos

Um conjunto de conectivos C é **completo** se, para qualquer função de verdade $f : \{V, F\}^n \rightarrow \{V, F\}$, existe uma fórmula ϕ com n variáveis, usando só conectivos de C , tal que ϕ **induz** exatamente f (isto é, $F_\phi = f$, onde $F_\phi(x_1, \dots, x_n) = v_\chi(\phi)$ para $v_\chi(p_i) = x_i$). Por outras palavras: C chega para exprimir *qualquer* função booleana possível.

Prova de que $\{\wedge, \vee, \neg\}$ é completo (dos slides, por indução em n , o número de variáveis):

- **Base ($n = 1$):** há exatamente 4 funções de verdade de aridade 1 (uma tabela com 2 linhas tem $2^2 = 4$ combinações possíveis de saída). Para cada uma, exhibe-se uma fórmula que a realiza: $\$f_1 = \$$ “sempre F” $\rightarrow \phi_1 = p \wedge \neg p$; $f_2 = \neg p \rightarrow \phi_2 = \neg p$; $f_3 = p \rightarrow \phi_3 = p$; $f_4 =$ “sempre V” $\rightarrow \phi_4 = p \vee \neg p$.
- **Passo indutivo:** assumindo que a propriedade vale para n , seja $f : \{V, F\}^{n+1} \rightarrow \{V, F\}$. Define-se $f_1(x_1, \dots, x_n) = f(x_1, \dots, x_n, V)$ e $f_2(x_1, \dots, x_n) = f(x_1, \dots, x_n, F)$ (fixando a última variável). Por hipótese de indução existem ϕ_1, ϕ_2 com $F_{\phi_i} = f_i$. Toma-se $\phi = (p_{n+1} \wedge \phi_1) \vee (\neg p_{n+1} \wedge \phi_2)$: quando $x_{n+1} = V$, ϕ reduz-se a ϕ_1 ; quando $x_{n+1} = F$, reduz-se a ϕ_2 — exatamente o comportamento de f .

Como consequência imediata (usando as leis de De Morgan e a lei da implicação para reescrever $\wedge$ e $\vee$), também $\{\neg, \rightarrow\}$ é completo: $\phi \wedge \psi \Leftrightarrow \neg(\phi \rightarrow \neg\psi)$ e $\phi \vee \psi \Leftrightarrow \neg\phi \rightarrow \psi$.

(Na Aula 3 aparece uma segunda prova, mais direta, do mesmo resultado: a construção da **forma normal disjuntiva** a partir da tabela de verdade.)

Exemplo resolvido — Exercício 1.19(a) (lab, proplogic.pdf) — negação, disjunção é completo

Mostrar que $p \rightarrow q$, $p \leftrightarrow q$ e $p \wedge q$ se escrevem só com $\neg$ e $\vee$ (a mesma técnica da prova acima, mas explicitada concretamente):

- $p \rightarrow q \Leftrightarrow \neg p \vee q$ (é diretamente a lei da implicação).
- $p \wedge q \Leftrightarrow \neg(\neg p \vee \neg q)$ (De Morgan aplicado ao contrário: $\neg\neg(p \wedge q) \Leftrightarrow \neg(\neg p \vee \neg q)$, e $\neg\neg(p \wedge q) \Leftrightarrow p \wedge q$).
- $p \leftrightarrow q \Leftrightarrow (p \rightarrow q) \wedge (q \rightarrow p)$. Usando os dois pontos anteriores para reescrever cada peça só com $\neg, \vee$: $p \rightarrow q \Leftrightarrow \neg p \vee q$ e $q \rightarrow p \Leftrightarrow \neg q \vee p$; e a conjunção dessas duas, pelo ponto anterior ($\phi \wedge \psi \Leftrightarrow \neg(\neg\phi \vee \neg\psi)$), fica:

$$p \leftrightarrow q \Leftrightarrow \neg(\neg(\neg p \vee q) \vee \neg(\neg q \vee p))$$

— uma fórmula só com $\neg$ e $\vee$, como pedido.

Exemplo resolvido — Exercício 1.19(b) (lab, proplogic.pdf) — NAND é universal

Define-se um novo conectivo $\bar{\wedge}$ (“NAND”) pela tabela: $V\bar{\wedge}V = F$; $V\bar{\wedge}F = F\bar{\wedge}V = F\bar{\wedge}F = V$ (ou seja, $p\bar{\wedge}q \Leftrightarrow \neg(p \wedge q)$: só é falso quando ambos são verdadeiros). Mostrar que $\neg p$, $p \wedge q$ e $p \vee q$ se escrevem só com $\bar{\wedge}$:

Negação: $p\bar{\wedge}p \Leftrightarrow \neg(p \wedge p) \Leftrightarrow \neg p$ (idempotência). Logo $\neg p = p\bar{\wedge}p$.

Conjunção: $p\bar{\wedge}q \Leftrightarrow \neg(p \wedge q)$, logo $\neg(p\bar{\wedge}q) \Leftrightarrow p \wedge q$. E $\neg X = X\bar{\wedge}X$ (pelo caso anterior, com $X = p\bar{\wedge}q$), logo:

$$p \wedge q = (p\bar{\wedge}q) \bar{\wedge} (p\bar{\wedge}q)$$

Disjunção: por De Morgan, $p \vee q \Leftrightarrow \neg(\neg p \wedge \neg q) \Leftrightarrow \neg p\bar{\wedge}\neg q$ (a própria definição de $\bar{\wedge}$ já é a negação de um $\wedge$). Substituindo $\neg p = p\bar{\wedge}p$ e $\neg q = q\bar{\wedge}q$:

$$p \vee q = (p\bar{\wedge}p) \bar{\wedge} (q\bar{\wedge}q)$$

Como $\{\neg, \wedge, \vee\}$ já é completo e todos os três se escrevem só com $\bar{\wedge}$, conclui-se que $\{\bar{\wedge}\}$ **sozinho já é um conjunto completo de conectivos** — é o princípio por trás dos circuitos NAND em eletrônica digital (qualquer circuito lógico pode ser construído só com portas NAND).

Aplicação: modelar problemas em lógica proposicional

Os problemas seguintes não introduzem teoria nova — mostram como usar tudo o que já foi visto (tradução para fórmulas, tabelas de verdade, satisfazibilidade de conjuntos) para resolver problemas de raciocínio “do mundo real”. A técnica chave nos problemas de cavaleiros/vilões é a **auto-referência**: se a representa “A é cavaleiro” (diz sempre a verdade) e S é a fórmula que traduz a frase que A disse, então tem de valer $a \leftrightarrow S$ — A é cavaleiro sse o que disse é verdade.

Exemplo resolvido — Exercício 1.10(a) (lab, proplogic.pdf) — ilha dos cavaleiros e vilões

A diz: “Pelo menos um de nós $[A, B]$ é vilão.” O que são A e B?

Seja a = “A é cavaleiro”, b = “B é cavaleiro”. A frase de A traduz-se por $S = \neg a \vee \neg b$. A condição de auto-referência é $a \leftrightarrow S$, isto é $a \leftrightarrow (\neg a \vee \neg b)$. Testam-se as 4 valorações de (a, b) :

a	b	$S = \neg a \vee \neg b$	$a \leftrightarrow S$
V	V	F	F
V	F	V	V
F	V	V	F
F	F	V	F

Só a linha $a = V, b = F$ é consistente (só nessa a biconditional dá V; nas outras a suposição contradiz-se a si própria). **A é cavaleiro, B é vilão.**

Exemplo resolvido — Exercício 1.10(e) (lab, proplogic.pdf) — três pessoas

A, B, C. A diz: “Somos todos vilões.” B diz: “Exatamente um de nós é cavaleiro.” O que são A, B e C? Sejam a, b, c os indicadores “é cavaleiro” de A, B, C. Traduzindo: $S_A = \neg a \wedge \neg b \wedge \neg c$ (todos vilões) e $S_B = (b \wedge \neg c) \vee (\neg b \wedge c)$ (exatamente um entre B e C é cavaleiro — note-se que S_B já não depende de a diretamente, mas a condição “exatamente um dos três” só se decompõe assim depois de se saber o valor de a ; para manter a tabela simples, calcula-se S_B como “exatamente um de $\{a, b, c\}$ ” diretamente nas 8 linhas).

Constrói-se a tabela com as duas condições de auto-referência, $\text{cond}_1 = (a \leftrightarrow S_A)$ e $\text{cond}_2 = (b \leftrightarrow S_B)$, para as 8 valorações de (a, b, c) — só interessam as linhas onde **ambas** são V:

a	b	c	S_A	cond_1	S_B (exat. 1 de a,b,c)	cond_2
V	V	V	F	F	F	F
V	V	F	F	F	F	F
V	F	V	F	F	F	V
V	F	F	F	F	V	F
F	V	V	F	V	F	F
F	V	F	F	V	V	V
F	F	V	F	V	F	F
F	F	F	V	F	V	F

Só a linha $a = F, b = V, c = F$ tem $\text{cond}_1 = V$ e $\text{cond}_2 = V$ simultaneamente. **A é vilão, B é cavaleiro, C é vilão** — e é a única solução consistente com as duas afirmações.

Exemplo resolvido — Exercício 1.11 (lab, proplogic.pdf) — suspeitos de um crime

Três suspeitos, A, B, C, fazem declarações: A: “B é culpado, mas C é inocente.” B: “Se A é culpado, então C é culpado.” C: “Eu sou inocente, mas um dos outros dois é culpado.”

Sejam g_a, g_b, g_c = “A/B/C é culpado”. Traduzindo cada declaração:

$$S_A = g_b \wedge \neg g_c \quad S_B = g_a \rightarrow g_c \quad S_C = \neg g_c \wedge (g_a \vee g_b)$$

Constrói-se **uma única tabela de verdade** com as 8 combinações de (g_a, g_b, g_c) e as três colunas S_A, S_B, S_C — e todas as alíneas seguintes respondem-se lendo linhas/colunas desta mesma tabela, sem repetir cálculos:

g_a	g_b	g_c	$S_A = g_b \wedge \neg g_c$	$S_B = g_a \rightarrow g_c$	$S_C = \neg g_c \wedge (g_a \vee g_b)$
V	V	V	F	V	F
V	V	F	V	F	V
V	F	V	F	V	F
V	F	F	F	F	V
F	V	V	F	V	F
F	V	F	V	V	V
F	F	V	F	V	F
F	F	F	F	V	F

(a) **As três declarações são compatíveis?** Pergunta-se se $\{S_A, S_B, S_C\}$ é satisfazível. Com $v(g_a) = F, v(g_b) = V, v(g_c) = F$: $\models_v S_A$ ($\models_v g_b, \not\models_v g_c$), $\models_v S_B$ ($\not\models_v g_a$) e $\models_v S_C$ ($\not\models_v g_c, \models_v g_b$). **Sim, são compatíveis**, e pela tabela essa é a **única** valoração que as satisfaz (só B culpado).

Exemplo resolvido — Exercício 1.11 (cont.) — consequência

(b) **Alguma declaração é consequência das outras duas?** Há três consequências para testar.

$\{S_A, S_B\} \models S_C$: *sim*. Seja v tal que $\models_v S_A$ e $\models_v S_B$.

- $\models_v g_b \wedge \neg g_c$ sse $\models_v g_b$ e $\not\models_v g_c$.
- $\models_v g_a \rightarrow g_c$ sse $\not\models_v g_a$ ou $\models_v g_c$. Como $\not\models_v g_c$, tem-se $\not\models_v g_a$.
- Então $\models_v \neg g_c$ e, como $\models_v g_b, \models_v g_a \vee g_b$. Logo $\models_v \neg g_c \wedge (g_a \vee g_b)$, isto é, $\models_v S_C$. ■

$\{S_B, S_C\} \models S_A$: *sim*. Seja v tal que $\models_v S_B$ e $\models_v S_C$.

- $\models_v \neg g_c \wedge (g_a \vee g_b)$ sse $\models_v g_c$ e $\models_v g_a \vee g_b$.
- $\models_v g_a \rightarrow g_c$ sse $\not\models_v g_a$ ou $\models_v g_c$. Como $\not\models_v g_c$, tem-se $\not\models_v g_a$.
- $\models_v g_a \vee g_b$ sse $\models_v g_a$ ou $\models_v g_b$. Como $\not\models_v g_a$, tem-se $\models_v g_b$.
- Logo $\models_v g_b \wedge \neg g_c$, isto é, $\models_v S_A$. ■

$\{S_A, S_C\} \not\models S_B$. Contraexemplo: $v(g_a) = V, v(g_b) = V, v(g_c) = F$ (2.^a linha da tabela).

- $\models_v g_b$ e $\not\models_v g_c$, logo $\models_v S_A$.
- $\not\models_v g_c$ e $\models_v g_a$, logo $\models_v S_C$.
- $\models_v g_a$ e $\not\models_v g_c$, logo $\not\models_v g_a \rightarrow g_c$, isto é, $\not\models_v S_B$.

Exemplo resolvido — Exercício 1.11 (cont.) — hipóteses sobre quem diz a verdade

(c) **Assumindo que os três são inocentes, quem mentiu?** Linha $g_a = F, g_b = F, g_c = F$: $S_A = F$ (A mentiu), $S_B = V$ (B disse a verdade), $S_C = F$ (C mentiu). **A e C mentiram; B disse a verdade.**

(d) **Assumindo que os três disseram a verdade, quem é inocente/culpado?** Precisa de $S_A = S_B = S_C = V$ — é exatamente a linha de (a): $g_a = F, g_b = V, g_c = F$. **Só B é culpado; A e C são inocentes.**

Exemplo resolvido — Exercício 1.11 (cont.) — inocentes dizem a verdade

(e) **Assumindo que o inocente disse a verdade e o culpado mentiu, quem é inocente/culpado?** Para cada pessoa X : X é inocente sse a sua declaração é verdadeira. Isso dá o conjunto

$$\Delta = \{ S_A \leftrightarrow \neg g_a, \quad S_B \leftrightarrow \neg g_b, \quad S_C \leftrightarrow \neg g_c \}$$

e procuram-se as v com $\models_v \Delta$. ($\models_v \phi \leftrightarrow \psi$ sse $\models_v \phi$ e $\models_v \psi$ têm o mesmo valor: ou ambos valem ou nenhum vale.)

Seja v tal que $\models_v \Delta$.

1. $\models_v g_a$. Suponha-se $\not\models_v g_a$. Então $\models_v \neg g_a$, logo $\models_v S_A$: $\models_v g_b$ e $\not\models_v g_c$. Como $\models_v g_b$, $\not\models_v \neg g_b$, logo $\not\models_v S_B$, isto é, $\models_v g_a$ e $\not\models_v g_c$. Mas supôs-se $\not\models_v g_a$. Contradição.
 2. $\models_v g_c$. Por 1, $\not\models_v \neg g_a$, logo $\not\models_v S_A$: $\not\models_v g_b$ ou $\models_v g_c$. Suponha-se $\not\models_v g_c$. Então $\not\models_v g_b$, logo $\models_v \neg g_b$ e $\models_v S_B$: $\not\models_v g_a$ ou $\models_v g_c$. Nenhuma das duas vale (por 1 e pela suposição). Contradição.
 3. $\not\models_v g_b$. Suponha-se $\models_v g_b$. Então $\not\models_v \neg g_b$, logo $\not\models_v S_B$: $\models_v g_a$ e $\not\models_v g_c$. Isto contradiz 2.
- Logo a única hipótese é $v(g_a) = V$, $v(g_b) = F$, $v(g_c) = V$. Verificação de que $\models_v \Delta$:

- $\not\models_v S_A$ (porque $\models_v g_c$) e $\not\models_v \neg g_a$. ✓
- $\models_v S_B$ (porque $\models_v g_c$) e $\models_v \neg g_b$. ✓
- $\not\models_v S_C$ (porque $\models_v g_c$) e $\not\models_v \neg g_c$. ✓

A e C são culpados; B é inocente. (Na tabela, é a única linha onde S_A , S_B e S_C coincidem com $\neg g_a$, $\neg g_b$ e $\neg g_c$.)

Atenção

Repara como (c), (d) e (e) dão **três respostas diferentes** ao mesmo enunciado — cada uma corresponde a uma hipótese diferente sobre quem diz a verdade. É um erro comum tentar decidir “quem é mesmo culpado” sem antes fixar qual das três hipóteses está a ser assumida; a tabela de verdade única (com as colunas S_A, S_B, S_C) é o que permite responder às três sem ambiguidade nem trabalho repetido.

Prática agora — Modelação (proplogic.pdf)

- **1.10 (g)** — raciocínio indireto diferente: não analisas o que A/B disseram, mas o que **C disse**.
- **1.10 (j)** — aqui és tu que **constróis** a pergunta certa (pista do enunciado: relaciona, por equivalência, o que queres saber com o tipo do habitante).
- **1.12** — mesma técnica de modelação do Exercício 1.11 (uma variável por cadeira, uma fórmula por regra).
- *Opcional:* **1.10 (k)**, que o professor chama “um problema invulgar”.

As restantes alíneas de 1.10 ((b), (c), (d), (f), (h), (i), (l)) repetem a técnica de (a)/(e) — não precisas de as fazer.

Soluções, uma alínea de cada vez: SOLUCOES_Logica.html

Aula 3 — Formas Normais e Satisfazibilidade

Até aqui, para saber se uma fórmula é satisfazível ou tautologia, a única ferramenta geral era a tabela de verdade — com 2^n linhas para n variáveis. Esta aula mostra que qualquer fórmula pode ser **transformada numa fórmula semanticamente equivalente com uma forma especial** (uma *forma normal*), e que, para algumas dessas formas, decidir a satisfazibilidade ou a validade passa a ser trivial (basta “olhar” para a fórmula). Algumas formas normais existem para **qualquer** fórmula (negativa, disjuntiva, conjuntiva); outras só existem para certas classes de fórmulas (Horn).

Literais e forma normal negativa (FNN)

Definição — Literal e forma normal negativa

Um **literal** é uma variável proposicional p ou a sua negação $\neg p$. (p diz-se literal **positivo**, $\neg p$ literal **negativo**.)

Uma fórmula está em **forma normal negativa** se a negação $\neg$ só aparece aplicada diretamente a variáveis, isto é, só dentro de literais. Ex: $(\neg p \wedge \neg q) \vee p$ está em FNN; $\neg(p \vee q)$ não está.

Definição — Proposição (existência da FNN)

Qualquer fórmula que só use os conectivos $\wedge$, $\vee$ e $\neg$ é semanticamente equivalente a uma fórmula em forma normal negativa.

Prova: basta aplicar repetidamente as **leis de De Morgan** (que “empurram” cada $\neg$ para dentro de um $\wedge/\vee$, trocando-o pelo outro) e eliminar as **duplas negações** ($\neg\neg\phi \Leftrightarrow \phi$) até cada $\neg$ ficar encostado a uma variável.

Exemplo resolvido — FNN de $\neg((p \vee q) \wedge \neg p)$ (slides)

$$\begin{aligned}
 & \neg((p \vee q) \wedge \neg p) \\
 \text{(De Morgan, no } \wedge \text{ exterior)} & \Leftrightarrow \neg(p \vee q) \vee \neg\neg p \\
 \text{(De Morgan, em } \neg(p \vee q)) & \Leftrightarrow (\neg p \wedge \neg q) \vee \neg\neg p \\
 \text{(dupla negação)} & \Leftrightarrow (\neg p \wedge \neg q) \vee p
 \end{aligned}$$

Em cada passo o $\neg$ desce um nível na árvore sintática; quando todos os $\neg$ estão encostados a variáveis, parou-se: está em FNN.

Atenção

Nota adicional (não estava explícito nos slides): a proposição só fala de fórmulas com $\wedge, \vee, \neg$. Se a fórmula tiver $\rightarrow$ (ou $\leftrightarrow$), **elimina-os primeiro**: $\phi \rightarrow \psi \Leftrightarrow \neg\phi \vee \psi$ e $\phi \leftrightarrow \psi \Leftrightarrow (\phi \rightarrow \psi) \wedge (\psi \rightarrow \phi)$. Só depois se empurram as negações. Esquecer este passo é o erro mais comum — $\neg(p \rightarrow q)$ **não** é “ $\neg p \rightarrow \neg q$ ”; é $p \wedge \neg q$ (ver Exercício 1.6(a) na Aula 2).

Forma normal disjuntiva (FND)

Definição — Forma normal disjuntiva

Uma fórmula está em **forma normal disjuntiva** se é uma **disjunção de conjunções de literais**:

$$(\alpha_{11} \wedge \dots \wedge \alpha_{1k_1}) \vee \dots \vee (\alpha_{n1} \wedge \dots \wedge \alpha_{nk_n})$$

onde cada α_{ij} é um literal. (“Um *ou* de *e*’s”).

Exemplos: $p \wedge q$ (uma só conjunção); $p \vee \neg p$ (duas conjunções, cada uma com um só literal); $(p \wedge q \wedge \neg r) \vee (\neg p \wedge r)$.

Há duas formas de obter uma FND: **a partir da tabela de verdade** (lema seguinte) ou **a partir da fórmula**, por equivalências.

FND a partir da tabela de verdade

Definição — Lema (toda a função de verdade tem uma FND)

Para qualquer função $f : \{V, F\}^n \rightarrow \{V, F\}$ existe uma fórmula ϕ em FND, com n variáveis, tal que $F_\phi = f$.

Prova (construtiva — é o próprio método):

- Se f dá F em todas as linhas, toma-se $\phi = p_1 \wedge \neg p_1$ (sempre falsa).
- Senão, para cada linha (avaliação) v da tabela, define-se o literal $l_i^v = p_i$ se $v(p_i) = V$ e $l_i^v = \neg p_i$ se $v(p_i) = F$, e a conjunção $\phi_v = l_1^v \wedge \dots \wedge l_n^v$.
- **Porquê isto funciona:** cada l_i^v é, por construção, verdadeiro em v , logo $v(\phi_v) = V$. E em qualquer **outra** linha v' , pelo menos uma variável tem valor diferente, o literal correspondente é falso, e ϕ_v é falsa. Ou seja: ϕ_v é **verdadeira exatamente na linha v e em mais nenhuma**.
- Toma-se $\phi = \bigvee_{f(v)=V} \phi_v$: a disjunção das ϕ_v das linhas em que f dá V . Então ϕ é verdadeira exatamente nessas linhas — ou seja, $F_\phi = f$.

Exemplo resolvido — FND a partir de uma tabela (slides)

linha	x_1	x_2	x_3	f	conjunção ϕ_v (só para as linhas com V)
1	V	V	V	V	$p_1 \wedge p_2 \wedge p_3$
2	V	V	F	V	$p_1 \wedge p_2 \wedge \neg p_3$
3	V	F	V	V	$p_1 \wedge \neg p_2 \wedge p_3$
4	V	F	F	F	—
5	F	V	V	V	$\neg p_1 \wedge p_2 \wedge p_3$
6	F	V	F	F	—
7	F	F	V	F	—
8	F	F	F	F	—

Regra para escrever cada conjunção: variável a V na linha $\rightarrow$ entra **positiva**; variável a F $\rightarrow$ entra **negada**. Ex.: linha 3 tem $x_1 = V, x_2 = F, x_3 = V$, logo $p_1 \wedge \neg p_2 \wedge p_3$.

Juntando com $\vee$ as 4 conjunções das linhas com V :

$$(p_1 \wedge p_2 \wedge p_3) \vee (p_1 \wedge p_2 \wedge \neg p_3) \vee (p_1 \wedge \neg p_2 \wedge p_3) \vee (\neg p_1 \wedge p_2 \wedge p_3)$$

(Repara: f é a função “maioria” — é V quando pelo menos duas das três variáveis são V .)

Atenção

Este lema é **outra prova** de que $\{\wedge, \vee, \neg\}$ é um conjunto completo de conectivos (Aula 2, “Conjuntos completos de conectivos”): a fórmula construída só usa $\wedge$, $\vee$ e $\neg$ e realiza qualquer f . Lá a prova era por indução no número de variáveis; aqui é direta, linha a linha. Ambas chegam ao mesmo resultado.

FND a partir de uma fórmula**Definição — Corolário (toda a fórmula tem uma FND equivalente)**

Qualquer fórmula é semanticamente equivalente a uma fórmula em FND. Para a obter:

1. eliminar $\rightarrow$ e $\leftrightarrow$, ficando só com $\wedge$, $\vee$, $\neg$;
2. passar para **forma normal negativa** (De Morgan + dupla negação);
3. aplicar a **distributividade do $\wedge$ sobre o $\vee$** até não haver nenhum $\vee$ dentro de um $\wedge$:

$$(\phi \vee \psi) \wedge \theta \Leftrightarrow (\phi \wedge \theta) \vee (\psi \wedge \theta) \quad \theta \wedge (\phi \vee \psi) \Leftrightarrow (\theta \wedge \phi) \vee (\theta \wedge \psi)$$

Exemplo resolvido — FND de $(p \vee r) \leftrightarrow (q \wedge \neg p)$, parte 1: passos 1 e 2 (slides)

Passo 1 — eliminar $\leftrightarrow$ e $\rightarrow$:

$$\begin{aligned} & (p \vee r) \leftrightarrow (q \wedge \neg p) \\ \Leftrightarrow & ((p \vee r) \rightarrow (q \wedge \neg p)) \wedge ((q \wedge \neg p) \rightarrow (p \vee r)) \quad (\leftrightarrow \text{ como duas implicações}) \\ \Leftrightarrow & (\neg(p \vee r) \vee (q \wedge \neg p)) \wedge (\neg(q \wedge \neg p) \vee (p \vee r)) \quad (\text{implicação, 2 vezes}) \end{aligned}$$

Passo 2 — forma normal negativa. No lado esquerdo, $\neg(p \vee r) \Leftrightarrow \neg p \wedge \neg r$ (De Morgan). No lado direito, $\neg(q \wedge \neg p) \Leftrightarrow \neg q \vee \neg\neg p \Leftrightarrow \neg q \vee p$ (De Morgan + dupla negação). Fica:

$$\underbrace{((\neg p \wedge \neg r) \vee (q \wedge \neg p))}_X \wedge \underbrace{((\neg q \vee p) \vee (p \vee r))}_{A \vee B}$$

com $A = \neg q \vee p$ e $B = p \vee r$. Isto já está em FNN, mas **não** em FND: há $\vee$ dentro do $\wedge$ principal.

Exemplo resolvido — FND de $(p \vee r) \leftrightarrow (q \wedge \neg p)$, parte 2: distributividade (slides)

Passo 3a — $X \wedge (A \vee B) \Leftrightarrow (X \wedge A) \vee (X \wedge B)$:

$$(((\neg p \wedge \neg r) \vee (q \wedge \neg p)) \wedge A) \vee (((\neg p \wedge \neg r) \vee (q \wedge \neg p)) \wedge B)$$

Passo 3b — em cada um dos dois blocos, $X = C_1 \vee C_2$ (com $C_1 = \neg p \wedge \neg r$, $C_2 = q \wedge \neg p$) volta a distribuir: $(C_1 \vee C_2) \wedge A \Leftrightarrow (C_1 \wedge A) \vee (C_2 \wedge A)$, e o mesmo com B :

$$(\neg p \wedge \neg r \wedge (\neg q \vee p)) \vee (q \wedge \neg p \wedge (\neg q \vee p)) \vee (\neg p \wedge \neg r \wedge (p \vee r)) \vee (q \wedge \neg p \wedge (p \vee r))$$

Passo 3c — cada um dos 4 termos ainda tem um $\vee$ lá dentro; distribui-se cada um $(K \wedge (a \vee b) \Leftrightarrow (K \wedge a) \vee (K \wedge b))$:

- **Termo 1** — $K = \neg p \wedge \neg r$, $(a \vee b) = \neg q \vee p$: $(\neg p \wedge \neg r \wedge \neg q) \vee (\neg p \wedge \neg r \wedge p)$
- **Termo 2** — $K = q \wedge \neg p$, $(a \vee b) = \neg q \vee p$: $(q \wedge \neg p \wedge \neg q) \vee (q \wedge \neg p \wedge p)$
- **Termo 3** — $K = \neg p \wedge \neg r$, $(a \vee b) = p \vee r$: $(\neg p \wedge \neg r \wedge p) \vee (\neg p \wedge \neg r \wedge r)$
- **Termo 4** — $K = q \wedge \neg p$, $(a \vee b) = p \vee r$: $(q \wedge \neg p \wedge p) \vee (q \wedge \neg p \wedge r)$

A FND é a disjunção destas 8 conjunções:

$$(\neg p \wedge \neg r \wedge \neg q) \vee (\neg p \wedge \neg r \wedge p) \vee (q \wedge \neg p \wedge \neg q) \vee (q \wedge \neg p \wedge p) \vee (\neg p \wedge \neg r \wedge p) \vee (\neg p \wedge \neg r \wedge r) \vee (q \wedge \neg p \wedge p) \vee (q \wedge \neg p \wedge r)$$

Satisfazibilidade de uma FND

Definição — Lema e corolário

Lema. Uma conjunção de literais $l_1 \wedge \dots \wedge l_n$ é satisfazível **sse** não contém um par complementar, isto é, para todos os i, j , l_i não é $\neg l_j$. (Se não há par $p/\neg p$, basta pôr cada literal positivo a V e cada negativo a F — não há conflito. Se há p e $\neg p$, nunca podem ser ambos verdadeiros.)

Corolário. Uma fórmula em FND é satisfazível **sse alguma** das suas conjunções é satisfazível (uma disjunção é verdadeira se **um** dos termos for).

Exemplo resolvido

- $p \wedge \neg q \wedge \neg r \wedge q$: contém $\neg q$ e $q \rightarrow$ **não** é satisfazível.
- $\neg p \wedge q \wedge \neg r \wedge \neg s$: nenhum par complementar $\rightarrow$ **é** satisfazível (com $p = F$, $q = V$, $r = F$, $s = F$).

Exemplo resolvido — $(p \vee r) \leftrightarrow (q \wedge \neg p)$ é satisfazível? (slides)

Usa-se a FND obtida acima e verifica-se **cada** uma das 8 conjunções:

#	conjunção	par complementar?	satisfazível?
1	$\neg p \wedge \neg r \wedge \neg q$	nenhum	sim
2	$\neg p \wedge \neg r \wedge p$	$\neg p, p$	não
3	$q \wedge \neg p \wedge \neg q$	$q, \neg q$	não
4	$q \wedge \neg p \wedge p$	$\neg p, p$	não
5	$\neg p \wedge \neg r \wedge p$	$\neg p, p$	não
6	$\neg p \wedge \neg r \wedge r$	$\neg r, r$	não
7	$q \wedge \neg p \wedge p$	$\neg p, p$	não
8	$q \wedge \neg p \wedge r$	nenhum	sim

Há (pelo menos) uma conjunção satisfazível $\rightarrow$ a fórmula **é satisfazível**. A conjunção 1 dá a valoração $p = q = r = F$; a 8 dá $p = F$, $q = r = V$.

Como as conjunções insatisfazíveis são sempre falsas, podem ser **apagadas** da disjunção sem mudar nada ($\phi \vee F \leftrightarrow \phi$), e fica a FND simplificada:

$$(p \vee r) \leftrightarrow (q \wedge \neg p) \Leftrightarrow (\neg p \wedge \neg q \wedge \neg r) \vee (\neg p \wedge q \wedge r)$$

Verificação direta (para confirmar que não houve engano): se $p = V$, o lado esquerdo $p \vee r$ é V e o direito $q \wedge \neg p$ é $F \rightarrow$ a equivalência é F . Se $p = F$, fica $r \leftrightarrow q$, que é V só para $q = r = F$ ou $q = r = V$. São exatamente as duas conjunções que sobraram. ✓

Atenção

Nota adicional (dos apontamentos da Prof.^a Nelma Moreira, em **Teóricas/**): se a fórmula **já está** em FND, este teste é **linear** no tamanho da fórmula — muito melhor do que os 2^n da tabela de verdade. Mas **atenção**: obter a FND pode ser caro — cada distributividade duplica parte da fórmula (o exemplo acima passou de uma fórmula pequena para 8 conjunções), e no pior caso a FND tem tamanho exponencial. Não se conhece nenhum algoritmo polinomial para a satisfazibilidade de uma fórmula qualquer (problema **SAT**); encontrar um responderia à pergunta “ $P = NP$?”. O que há são classes de fórmulas em que o problema é fácil (FND, Horn) e algoritmos que na prática funcionam bem para fórmulas em FNC.

Forma normal conjuntiva (FNC)

Definição — Forma normal conjuntiva

Uma fórmula está em **forma normal conjuntiva** se é uma **conjunção de disjunções de literais** (“um *e* de *ou*’s”):

$$(\alpha_{11} \vee \dots \vee \alpha_{1k_1}) \wedge \dots \wedge (\alpha_{n1} \vee \dots \vee \alpha_{nk_n})$$

Cada disjunção de literais chama-se uma **cláusula**. (Na Aula 4 as cláusulas passam a escrever-se como **conjuntos de literais**; ver a secção “Cláusulas” dessa aula.)

Definição — Lema dual (tautologia de uma FNC)

Por **dualidade** com o lema da FND: uma disjunção de literais $l_1 \vee \dots \vee l_n$ é **tautologia** sse **contém um par complementar** (algum l_i é $\neg l_j$: se tem $p \vee \neg p$ lá dentro, é sempre verdadeira; se não tem, pondo cada literal a *F* ela falha).

Logo, uma fórmula em FNC é **tautologia** sse **todas as suas cláusulas são tautologias** (uma conjunção é sempre verdadeira sse cada termo é sempre verdadeiro).

Frequente em exame

Guarda a **simetria**: FND serve para decidir **satisfazibilidade** (basta **uma** conjunção sem par complementar); FNC serve para decidir se é **tautologia** (**todas** as cláusulas têm de ter um par complementar). O contrário **não** é fácil: decidir se uma FNC é satisfazível é exatamente o problema SAT difícil.

Duas formas de obter uma FNC — as mesmas duas da FND, “ao contrário”:

1. **A partir da tabela de verdade**, pelo método dual: escolher as linhas em que f dá **F**; para cada uma, formar a **disjunção** em que a variável entra **negada se está a V** nessa linha e **positiva se está a F**; tomar a **conjunção** dessas disjunções.
2. **A partir da fórmula**: passos 1 e 2 iguais aos da FND, mas no passo 3 usa-se a distributividade do $\vee$ sobre o $\wedge$:

$$(\phi \wedge \psi) \vee \theta \Leftrightarrow (\phi \vee \theta) \wedge (\psi \vee \theta) \quad \theta \vee (\phi \wedge \psi) \Leftrightarrow (\theta \vee \phi) \wedge (\theta \vee \psi)$$

Exemplo resolvido — FNC a partir de uma tabela (slides; a mesma função da FND acima)

Agora interessam as linhas com **F** (4, 6, 7, 8). Para cada uma, a cláusula é construída para ser **falsa exatamente nessa linha**: cada literal tem de ser *F* ali, por isso variável a $V \rightarrow$ entra **negada**, variável a $F \rightarrow$ entra **positiva**.

linha	x_1	x_2	x_3	f	cláusula
4	V	F	F	F	$\neg p_1 \vee p_2 \vee p_3$
6	F	V	F	F	$p_1 \vee \neg p_2 \vee p_3$
7	F	F	V	F	$p_1 \vee p_2 \vee \neg p_3$
8	F	F	F	F	$p_1 \vee p_2 \vee p_3$

Verificação da linha 4: com $x_1 = V, x_2 = F, x_3 = F$, $\neg p_1 \vee p_2 \vee p_3 = F \vee F \vee F = F$; e em qualquer outra linha pelo menos um literal é *V*. A conjunção das 4 cláusulas é *F* exatamente nas linhas 4, 6, 7, 8 — ou seja, realiza f :

$$(\neg p_1 \vee p_2 \vee p_3) \wedge (p_1 \vee \neg p_2 \vee p_3) \wedge (p_1 \vee p_2 \vee \neg p_3) \wedge (p_1 \vee p_2 \vee p_3)$$

Exemplo resolvido — FNC de $(p \vee r) \leftrightarrow (q \wedge \neg p)$ e é tautologia? (slides)

Os passos 1 e 2 são **exatamente** os da FND (ver acima), e dão:

$$((\neg p \wedge \neg r) \vee (q \wedge \neg p)) \wedge ((\neg q \vee p) \vee (p \vee r))$$

O lado direito já é uma cláusula: $\neg q \vee p \vee p \vee r$ (associatividade). Falta o lado esquerdo, que é um $\vee$ de $\wedge$'s.

Passo 3a — distribuir com $\theta = \neg p \wedge \neg r$, $\phi = q$, $\psi = \neg p$, usando $\theta \vee (\phi \wedge \psi) \Leftrightarrow (\theta \vee \phi) \wedge (\theta \vee \psi)$:

$$(\neg p \wedge \neg r) \vee (q \wedge \neg p) \Leftrightarrow ((\neg p \wedge \neg r) \vee q) \wedge ((\neg p \wedge \neg r) \vee \neg p)$$

Passo 3b — cada bloco ainda tem um $\wedge$ dentro de um $\vee$; distribui-se outra vez $((a \wedge b) \vee c \Leftrightarrow (a \vee c) \wedge (b \vee c))$:

- $(\neg p \wedge \neg r) \vee q \Leftrightarrow (\neg p \vee q) \wedge (\neg r \vee q)$
- $(\neg p \wedge \neg r) \vee \neg p \Leftrightarrow (\neg p \vee \neg p) \wedge (\neg r \vee \neg p)$

FNC final (5 cláusulas):

$$(\neg p \vee q) \wedge (\neg r \vee q) \wedge (\neg p \vee \neg p) \wedge (\neg r \vee \neg p) \wedge (\neg q \vee p \vee p \vee r)$$

É tautologia? Verifica-se cada cláusula: $\neg p \vee q$ não tem par complementar $\rightarrow$ já não é tautologia, e a fórmula também **não** é (resposta dos slides: **Não**). Contraexemplo tirado da própria cláusula (pôr os seus literais a F): $p = V$, $q = F$ — de facto, com $p = V$ o lado $p \vee r$ é V e $q \wedge \neg p$ é F , logo a equivalência é F . ✓

Atenção

Para “limpar” uma FND/FNC usam-se as leis extra da Aula 2 (complementaridade, elemento neutro/absorvente, absorção). Na FNC acima, por exemplo: $\neg p \vee \neg p \Leftrightarrow \neg p$ (idempotência), e a cláusula $\neg r \vee \neg p$ é absorvida por $\neg p$. Fica $(\neg p \vee q) \wedge (\neg r \vee q) \wedge \neg p \wedge (\neg q \vee p \vee r)$ — com $p = F$ forçado, sobra $(\neg r \vee q) \wedge (\neg q \vee r)$, ou seja $q \leftrightarrow r$: coerente com a verificação feita na FND. Uma FND/FNC não é única; qualquer uma equivalente está certa.

Aplicação aos exercícios da prática: FND/FNC

Exemplo resolvido — Exercício 1.17(a) (lab, proplogic.pdf) — FND e FNC a partir da tabela

A função (a) tem esta coluna (na ordem de linhas do enunciado, p, q, r de VVV até FFF): V, V, V, V, F, V, V, V . Só **uma** linha dá F : a linha $p = F, q = V, r = V$.

FNC (uma cláusula por linha com F — só há uma): na linha $p = F, q = V, r = V$, p está a $F \rightarrow$ entra positivo; q e r estão a $V \rightarrow$ entram negados:

$$p \vee \neg q \vee \neg r$$

FND (uma conjunção por linha com V — há sete):

p	q	r	conjunção
V	V	V	$p \wedge q \wedge r$
V	V	F	$p \wedge q \wedge \neg r$
V	F	V	$p \wedge \neg q \wedge r$
V	F	F	$p \wedge \neg q \wedge \neg r$
F	V	F	$\neg p \wedge q \wedge \neg r$
F	F	V	$\neg p \wedge \neg q \wedge r$
F	F	F	$\neg p \wedge \neg q \wedge \neg r$

$$(p \wedge q \wedge r) \vee (p \wedge q \wedge \neg r) \vee (p \wedge \neg q \wedge r) \vee (p \wedge \neg q \wedge \neg r) \vee (\neg p \wedge q \wedge \neg r) \vee (\neg p \wedge \neg q \wedge r) \vee (\neg p \wedge \neg q \wedge \neg r)$$

Lição: quando a tabela tem **poucos** F , a FNC sai muito mais curta; quando tem **poucos** V , é a FND. Ambas estão certas — as duas representam a mesma função (confirma: a FNC só falha em $p = F, q = V, r = V$, que é a única linha que falta na FND). Também se podia simplificar a FND de 7 termos, mas não é pedido.

Exemplo resolvido — Exercício 1.18(d) (lab, proplogic.pdf) — FND e FNC de $(p \rightarrow q) \rightarrow (\neg p \rightarrow \neg q)$

$$\begin{aligned} & (p \rightarrow q) \rightarrow (\neg p \rightarrow \neg q) \\ \Leftrightarrow & \neg(p \rightarrow q) \vee (\neg p \rightarrow \neg q) && \text{(implicação, na } \rightarrow \text{ exterior)} \\ \Leftrightarrow & \neg(\neg p \vee q) \vee (\neg \neg p \vee \neg q) && \text{(implicação, nas duas interiores)} \\ \Leftrightarrow & (\neg \neg p \wedge \neg q) \vee (\neg \neg p \vee \neg q) && \text{(De Morgan)} \\ \Leftrightarrow & (p \wedge \neg q) \vee p \vee \neg q && \text{(dupla negação, 2 vezes; associatividade)} \end{aligned}$$

FND: $(p \wedge \neg q) \vee p \vee \neg q$ já é FND — três conjunções: $p \wedge \neg q$, p (conjunção de um só literal), $\neg q$. Simplificando por absorção $(p \vee (p \wedge \neg q) \Leftrightarrow p)$:

$$p \vee \neg q$$

FNC: distribui-se o $\vee$ sobre o $\wedge$ em $(p \wedge \neg q) \vee (p \vee \neg q)$, com $\theta = p \vee \neg q$:

$$(p \vee p \vee \neg q) \wedge (\neg q \vee p \vee \neg q) \Leftrightarrow (p \vee \neg q) \wedge (p \vee \neg q) \Leftrightarrow p \vee \neg q$$

(idempotência duas vezes). Repara que $p \vee \neg q$ é **ao mesmo tempo** FND (duas conjunções de um literal) e FNC (uma cláusula) — é normal com fórmulas pequenas.

Não é tautologia (a cláusula não tem par complementar): com $p = F, q = V$, $p \rightarrow q = V$ e $\neg p \rightarrow \neg q = V \rightarrow F = F$. (A alínea (k) do enunciado é **igual** a esta.)

Exemplo resolvido — Exercício 1.18(h) (lab, proplogic.pdf) — FND e FNC de $(p \rightarrow q) \wedge (\neg q \rightarrow (p \vee \neg q))$

Eliminar implicações: $p \rightarrow q \Leftrightarrow \neg p \vee q$; e $\neg q \rightarrow (p \vee \neg q) \Leftrightarrow \neg\neg q \vee p \vee \neg q \Leftrightarrow q \vee p \vee \neg q$.

$$(\neg p \vee q) \wedge (q \vee p \vee \neg q)$$

FNC: isto já é uma FNC (conjunção de duas cláusulas). Repara que a segunda cláusula contém q e $\neg q$: é tautologia, logo vale sempre V e pode sair $(\phi \wedge V \Leftrightarrow \phi)$. FNC simplificada: $\neg p \vee q$.

FND (fazendo a distribuição toda, sem simplificar antes, para treinar): $(\neg p \vee q) \wedge X$ com $X = q \vee p \vee \neg q$ distribui como $(\neg p \wedge X) \vee (q \wedge X)$, e cada um distribui outra vez sobre os 3 literais de X :

$$(\neg p \wedge q) \vee (\neg p \wedge p) \vee (\neg p \wedge \neg q) \vee (q \wedge q) \vee (q \wedge p) \vee (q \wedge \neg q)$$

Apagam-se as conjunções com par complementar $(\neg p \wedge p \text{ e } q \wedge \neg q)$ e $q \wedge q \Leftrightarrow q$:

$$(\neg p \wedge q) \vee (\neg p \wedge \neg q) \vee q \vee (p \wedge q)$$

Isto é uma FND correta. Simplificando por absorção (q absorve $\neg p \wedge q$ e $p \wedge q$), sobra $(\neg p \wedge \neg q) \vee q$; e como $(\neg p \wedge \neg q) \vee q \Leftrightarrow (\neg p \vee q) \wedge (\neg q \vee q) \Leftrightarrow \neg p \vee q$, chega-se ao mesmo $\neg p \vee q$ da FNC — que também é FND. ✓

Lição: detetar cedo uma cláusula-tautologia (ou uma conjunção contraditória) poupa imenso trabalho de distribuição.

Pratica agora — Formas normais (proplogic.pdf)

- **1.17 (b) e (c)** — FND e FNC pela tabela. Na (c), olha bem para a coluna antes de começar: há uma fórmula muito mais curta escondida (as formas canónicas saem longas, mas simplificam-se).
- **1.18 (e)** — é a Lei de Peirce (Aula 2): o que acontece à FNC de uma tautologia?
- **1.18 (l)** — a mais comprida, boa para treinar a distributividade com 4 variáveis.
- **1.20** — o programa: valoração, tabela, classificação e FNC/FND (junta tudo o que viste até aqui).

As restantes alíneas de 1.18 são a mesma técnica; não precisas de as fazer todas (e repara que (a) = (c) e (d) = (k) no próprio enunciado).

Soluções, uma alínea de cada vez: [SOLUCOES_Logica.html](#)

Fórmulas de Horn

Definição — Fórmula de Horn

Uma **fórmula de Horn** é uma fórmula em **FNC** em que **cada cláusula tem no máximo um literal positivo**.

Exemplos (dos slides), com o nº de literais positivos por cláusula:

- $p \wedge \neg q \wedge (q \vee \neg p)$: cláusulas p (1), $\neg q$ (0), $q \vee \neg p$ (1) $\rightarrow$ Horn.
- $(\neg p \vee \neg q \vee \neg s \vee p) \wedge (\neg q \vee \neg r \vee p) \wedge (\neg p \vee \neg s \vee s)$: 1, 1, 1 $\rightarrow$ Horn.
- $(\neg p \vee \neg q \vee \neg s) \wedge (\neg q \vee \neg r \vee p) \wedge s$: 0, 1, 1 $\rightarrow$ Horn.

Contra-exemplo: $(p \vee q) \wedge \neg r$ **não** é Horn — a cláusula $p \vee q$ tem dois literais positivos.

Cada cláusula de Horn pode ler-se como uma **implicação** (usando $\neg a \vee b \Leftrightarrow a \rightarrow b$ e De Morgan $\neg p_1 \vee \dots \vee \neg p_n \Leftrightarrow \neg(p_1 \wedge \dots \wedge p_n)$):

forma da cláusula	lida como implicação	nome informal
$\neg p_1 \vee \dots \vee \neg p_n \vee p$	$(p_1 \wedge \dots \wedge p_n) \rightarrow p$	regra
$\neg p_1 \vee \dots \vee \neg p_n$ (sem positivo)	$(p_1 \wedge \dots \wedge p_n) \rightarrow F$	restrição (“não podem ser todos V ”)

forma da cláusula	lida como implicação	nome informal
p (sem negativos)	$V \rightarrow p$	facto (p tem de ser V)

Atenção

Nem todas as fórmulas têm uma fórmula de Horn equivalente — basta que a sua FNC tenha uma cláusula com mais de um literal positivo que não possa ser simplificada (ex.: $p \vee q$). A vantagem das fórmulas de Horn é que a sua satisfazibilidade decide-se com um **algoritmo eficiente** (linear/polinomial), sem tabela de verdade.

Algoritmo de satisfazibilidade para fórmulas de Horn

A ideia: começar com todas as variáveis “desconhecidas” e só atribuir V a uma variável quando **for obrigatório** (quando, sem isso, uma cláusula ficaria falsa). Se essas atribuições obrigatórias fizerem uma cláusula ficar F , a fórmula é insatisfazível; senão, põem-se todas as restantes variáveis a F e está encontrada uma valoração que a satisfaz.

Definição — Algoritmo (versão dos slides)

1. Escrever numa linha as variáveis e a fórmula (como numa linha de tabela de verdade, ainda vazia).
2. Se alguma **variável sozinha** é um dos elementos da conjunção (um **facto**, $V \rightarrow p$), atribuir-lhe V . **Porquê?** Porque uma conjunção só é V se cada elemento for V ; se p é um elemento, tem de ser $p = V$ em qualquer valoração que satisfaça a fórmula.
3. Com os valores já conhecidos, avaliar cada cláusula. Se numa cláusula todos os literais negativos já estão a F (as suas variáveis estão a V), então, para a cláusula ser V , o **literal positivo** tem de ser $V \rightarrow$ atribuir V a essa variável.
4. Repetir o passo 3 até nada mais poder ser acrescentado.
5. Se algum elemento da conjunção ficou com valor F , a fórmula é $F \rightarrow$ **não satisfazível**. Caso contrário, é **satisfazível**: atribui-se F a todas as variáveis restantes e a fórmula fica V .

Exemplo resolvido — $p \wedge \neg q \wedge (q \vee \neg p)$ (slides)

Cláusulas: $C_1 = p$, $C_2 = \neg q$, $C_3 = q \vee \neg p$ (lida como $p \rightarrow q$).

Passo 1 — linha vazia:

$$\frac{p \quad q \quad C_1 = p \quad C_2 = \neg q \quad C_3 = q \vee \neg p}{}$$

Passo 2 — p é um elemento da conjunção (facto) $\rightarrow p = V$:

$$\frac{p \quad q \quad C_1 = p \quad C_2 = \neg q \quad C_3 = q \vee \neg p}{V \quad \quad V \quad \quad (\neg p = F)}$$

Passo 3 — em C_3 , o literal negativo $\neg p$ já é F ; para C_3 ser V , o positivo q tem de ser $V \rightarrow q = V$:

$$\frac{p \quad q \quad C_1 = p \quad C_2 = \neg q \quad C_3 = q \vee \neg p}{V \quad V \quad \quad V \quad \quad V}$$

Passo 4 — repete-se: agora $C_2 = \neg q$ com $q = V$ dá F :

$$\frac{p \quad q \quad C_1 = p \quad C_2 = \neg q \quad C_3 = q \vee \neg p}{V \quad V \quad \quad V \quad \quad \mathbf{F} \quad \quad V}$$

Passo 5 — um elemento da conjunção (C_2) ficou $F \rightarrow$ a fórmula é $F \rightarrow$ **não é satisfazível**. (Não há escolha possível: $p = V$ e $q = V$ foram ambos **obrigatórios**.)

Atenção

Nota adicional — **porque é que o algoritmo está correto** (é o Exercício 1.21(b) do lab; nos slides fica só o “porquê?”):

- **Se diz “insatisfazível”, está certo:** cada V atribuído foi *obrigatório* — qualquer valoração que satisfaça a fórmula tem de o ter. Se as atribuições obrigatórias já tornam uma cláusula F , nenhuma valoração satisfaz a fórmula.
- **Se diz “satisfazível”, está certo:** com as marcadas a V e as outras a F , pega-se numa cláusula qualquer. (i) Se o seu literal positivo está marcado, ela é V . (ii) Senão, algum dos seus literais negativos $\neg x$ tem x **não** marcado (se todos estivessem marcados, o passo 3 teria obrigado a marcar o positivo, ou — se não houver positivo — a cláusula seria F e o algoritmo teria parado no passo 5); logo $x = F$ e $\neg x = V$, e a cláusula é V .

É exatamente aqui que se usa “no máximo um literal positivo”: pôr as restantes a F torna verdadeiros **todos** os literais negativos por decidir, e nunca é preciso “escolher” entre dois positivos. Por isso nunca há retrocesso (*backtracking*) e o algoritmo é eficiente.

Exemplo resolvido — O mesmo algoritmo em pseudo-código (base para o Exercício 1.21(a))

Vendo cada cláusula como implicação $(p_1 \wedge \dots \wedge p_n) \rightarrow h$, com h uma variável ou F (e $n = 0$ para factos):

```

marcadas := {}
-- variáveis que TÊM de ser V
repetir
  mudou := falso
  para cada cláusula (p1 e ... e pn) -> h:
    se p1,...,pn estão todos em marcadas: -- antecedente todo V
      se h = F: devolver INSATISFAZÍVEL
      se h não está em marcadas:
        acrescentar h a marcadas; mudou := verdadeiro
até não mudou
devolver SATISFAZÍVEL -- v(x)=V se x em marcadas, senão v(x)=F

```

Cada iteração do **repetir** marca pelo menos uma variável nova (ou pára), por isso há no máximo (n° de variáveis + 1) iterações — polinomial.

Exemplo resolvido — Exercício 1.21(c), 1.^a e 2.^a fórmulas (lab, proplogic.pdf)

(i) $(\neg p \vee \neg q) \wedge (\neg q \vee r) \wedge q$. Como implicações: $(p \wedge q) \rightarrow F$, $q \rightarrow r$, $V \rightarrow q$.

- Facto $q \rightarrow q = V$.
- $q \rightarrow r$: antecedente q marcado $\rightarrow r = V$.
- $(p \wedge q) \rightarrow F$: antecedente precisa de p e q ; p não está marcado $\rightarrow$ não dispara.
- Nova volta: nada muda. Nenhuma cláusula ficou F .

Satisfazível, com $q = V$, $r = V$ e a restante $p = F$. Verificação: $\neg p \vee \neg q = V \vee F = V$; $\neg q \vee r = F \vee V = V$; $q = V$. ✓

(ii) $p \wedge (\neg p \vee q) \wedge (\neg q \vee p)$. Como implicações: $V \rightarrow p$, $p \rightarrow q$, $q \rightarrow p$.

- Facto $p \rightarrow p = V$.
- $p \rightarrow q$: p marcado $\rightarrow q = V$.
- $q \rightarrow p$: q marcado, mas p já estava marcado $\rightarrow$ nada novo.
- Nova volta: nada muda; nenhuma cláusula ficou F .

Satisfazível, com $p = q = V$ (não sobram variáveis para pôr a F). Verificação: $p = V$; $\neg p \vee q = F \vee V = V$; $\neg q \vee p = F \vee V = V$. ✓

Pratica agora — Horn (proplogic.pdf)

- **1.21 (c)**, 3.^a e 4.^a fórmulas: $\neg p \wedge (\neg p \vee q) \wedge \neg q$ (o que acontece quando **não há nenhum facto?**) e $p \wedge (\neg p \vee q) \wedge \neg r$.
- **1.21 (a)** — implementar o algoritmo a partir do pseudo-código acima.

Soluções, uma alínea de cada vez: [SOLUCOES_Logica.html](#)

Os slides da Aula 4 começam por rever as fórmulas de Horn e o algoritmo acima (é a mesma matéria, por isso está toda aqui). O exercício dos slides dessa aula, $p \wedge (\neg p \vee q) \wedge \neg r$, é a 4.^a fórmula da 1.21(c), na caixa acima.

Aula 4 — Satisfazibilidade: cláusulas e o algoritmo de Davis-Putnam

Os slides desta aula começam por rever as formas normais (negativa, disjuntiva, conjuntiva), a satisfazibilidade de uma FND e as fórmulas de Horn com o seu algoritmo. Isso já está explicado na Aula 3 (“Fórmulas de Horn”). A matéria nova é o caso **geral**: decidir se uma fórmula qualquer em FNC é satisfazível, com o **algoritmo de Davis-Putnam (DLL)**.

O problema SAT

Saber se uma fórmula da lógica proposicional é satisfazível é um problema **central na informática**, porque muitos problemas de outras áreas se podem escrever como uma fórmula e a pergunta “há solução?” passa a ser “a fórmula é satisfazível?”:

- **otimização**: planeamento, escalonamento de horários (*scheduling*), etc.;
- **combinatória**: coloração de grafos, etc.;
- **teorias da lógica de primeira ordem**: programação linear, aritmética de números reais, sequências de bits, apontadores, etc. (os *SMT solvers*).

Se a fórmula não está em FND, nem numa classe em que a satisfazibilidade seja fácil (como Horn), a tabela de verdade é o único caminho? **No pior caso, sim**: não se conhece nenhum algoritmo para a satisfazibilidade que não seja **exponencial** no número de variáveis e conectivos (**SAT é NP-completo**; ver também a nota da Aula 3 sobre “ $P = NP?$ ”).

Mas, **no caso geral**, há algoritmos muito melhores do que construir a tabela inteira. A ideia é **ir construindo uma valoração parcial** que satisfaça a fórmula, em vez de gerar valorações completas (linhas da tabela) e testar cada uma. Isto fica especialmente simples com fórmulas em **FNC**, escritas de forma compacta como conjuntos de **cláusulas**.

Cláusulas

Definição — Cláusula

Uma **cláusula** é uma disjunção de literais $l_1 \vee l_2 \vee \dots \vee l_n$, com $n \geq 0$ (é o mesmo nome da Aula 3, “Forma normal conjuntiva”). Representa-se pelo **conjunto** dos seus literais $\{l_1, \dots, l_n\}$.

- Se $n = 0$, a cláusula é **vazia** e corresponde a F .
- Se $n = 1$, a cláusula é **unitária**.

Exemplo: $p \vee \neg q \vee \neg p \vee s$ representa-se por $\{p, \neg q, \neg p, s\}$.

Qualquer fórmula em **FNC** representa-se por um **conjunto de cláusulas** (a conjunção fica implícita).

Exemplo resolvido

$$\neg p \wedge (q \vee r \vee q) \wedge (\neg r \vee \neg s) \wedge (p \vee s) \wedge (\neg q \vee \neg s)$$

corresponde ao conjunto de cláusulas

$$\{\{\neg p\}, \{q, r\}, \{\neg r, \neg s\}, \{p, s\}, \{\neg q, \neg s\}\}$$

Repara na segunda cláusula: $q \vee r \vee q$ fica $\{q, r\}$, porque um conjunto não tem repetidos (é a idempotência, $q \vee q \Leftrightarrow q$). Pela mesma razão, uma cláusula repetida no conjunto conta uma só vez (idempotência do $\wedge$).

Atenção — O que quer dizer um conjunto vazio

Nota adicional (não estava explícito nos slides, mas o algoritmo DLL abaixo depende disto): há dois “vazios” diferentes.

- **Cláusula vazia** $\{\}$ (escreve-se F): uma disjunção sem nenhum literal é **falsa**. Se um conjunto de cláusulas contém a cláusula vazia, é **insatisfazível** (uma conjunção com um F é F).
- **Conjunto de cláusulas vazio** $\{\}$: uma conjunção sem nenhum termo é **verdadeira** (não há nenhuma cláusula para falhar). É **satisfazível**.

Definição — Literal complementar

Dado um literal l , o **literal complementar** $\tilde{l}$ é:

$$\tilde{l} = \begin{cases} \neg l, & \text{se } l \text{ é uma variável (positivo)} \\ p, & \text{se } l \text{ é da forma } \neg p \text{ (negativo)} \end{cases}$$

Por exemplo, $\tilde{p} = \neg p$ e $\widetilde{\neg p} = p$.

Como se viu na Aula 3 (lema dual), uma cláusula é **tautologia** se contém um par de literais complementares p e $\neg p$. Essas cláusulas **podem ser retiradas** do conjunto sem alterar a satisfazibilidade: são sempre V , e $V \wedge \varphi \Leftrightarrow \varphi$. Por exemplo, $\{p, \neg q, \neg p, s\}$ acima pode simplesmente sair.

Propagação unitária

O algoritmo de **Davis-Putnam** tem uma versão inicial de **1960** e ainda é a base de muitos dos algoritmos mais eficientes de hoje (ver as competições de *SAT solvers*, <http://www.satcompetition.org/>). A ideia é **considerar os valores possíveis para cada variável e simplificar a fórmula** com essas atribuições, até se poder concluir se é satisfazível. A simplificação principal é a **propagação unitária**.

Definição — Propagação unitária

Seja S um conjunto de cláusulas. Obtém-se S' a partir de S por **propagação unitária** repetindo a seguinte transformação: se S contém uma **cláusula unitária** l , então

1. **retiram-se** de S todas as cláusulas da forma $l \vee C'$ (as que contêm l , incluindo a própria cláusula l);
2. **substitui-se** cada cláusula da forma $\tilde{l} \vee C'$ pela cláusula C' (tira-se o literal complementar).

Atenção — Porque é que a propagação não muda a satisfazibilidade

Nota adicional (não estava explícito nos slides). Se l é uma cláusula sozinha, qualquer valoração que satisfaça S tem de pôr $l = V$ (é um elemento da conjunção, como os factos de Horn). Com $l = V$:

- uma cláusula que contém l fica V de certeza: pode sair;
- numa cláusula que contém $\tilde{l}$, esse literal é F e não ajuda nada ($F \vee C' \Leftrightarrow C'$): pode sair da cláusula.

Se ao tirar $\tilde{l}$ uma cláusula fica **vazia** (era só $\tilde{l}$), aparece F : o conjunto é insatisfazível. Os literais propagados dizem também **que valores** têm as variáveis numa valoração que satisfaça S .

Exemplo resolvido — Exemplo dos slides: só com propagação (1/2)

$$S = \{p_1, \neg p_1 \vee \neg p_2, p_3 \vee p_2, \neg p_7 \vee p_2, \neg p_3 \vee p_4, \neg p_3 \vee p_5, \\ \neg p_4 \vee \neg p \vee q, \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p \vee p_7, \neg r \vee p_7\}$$

Propagar p_1 . Saem as cláusulas com p_1 : só a própria p_1 . Às que têm $\neg p_1$ tira-se esse literal: $\neg p_1 \vee \neg p_2$ fica $\neg p_2$.

$$\{\neg p_2, p_3 \vee p_2, \neg p_7 \vee p_2, \neg p_3 \vee p_4, \neg p_3 \vee p_5, \neg p_4 \vee \neg p \vee q, \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p \vee p_7, \neg r \vee p_7\}$$

Propagar $\neg p_2$. Sai $\neg p_2$. O complementar é p_2 : $p_3 \vee p_2$ fica p_3 e $\neg p_7 \vee p_2$ fica $\neg p_7$.

$$\{p_3, \neg p_7, \neg p_3 \vee p_4, \neg p_3 \vee p_5, \neg p_4 \vee \neg p \vee q, \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p \vee p_7, \neg r \vee p_7\}$$

Propagar p_3 . Sai p_3 . $\neg p_3 \vee p_4$ fica p_4 ; $\neg p_3 \vee p_5$ fica p_5 .

Propagar $\neg p_7$. Sai $\neg p_7$. $p \vee p_7$ fica p ; $\neg r \vee p_7$ fica $\neg r$.

$$\{p_4, p_5, \neg p_4 \vee \neg p \vee q, \neg p_5 \vee \neg p_6 \vee r, \neg p \vee \neg q \vee p_6, p, \neg r\}$$

Exemplo resolvido — Exemplo dos slides: só com propagação (2/2)

Propagar p_4 . $\neg p_4 \vee \neg p \vee q$ fica $\neg p \vee q$.

Propagar p_5 . $\neg p_5 \vee \neg p_6 \vee r$ fica $\neg p_6 \vee r$.

Propagar $\neg r$. $\neg p_6 \vee r$ fica $\neg p_6$.

Propagar p . Sai p . $\neg p \vee q$ fica q ; $\neg p \vee \neg q \vee p_6$ fica $\neg q \vee p_6$.

$$\{q, \neg p_6, \neg q \vee p_6\}$$

Propagar q . $\neg q \vee p_6$ fica p_6 : $\{\neg p_6, p_6\}$.

Propagar $\neg p_6$. Sai $\neg p_6$. A cláusula p_6 perde o seu único literal e fica **vazia**:

$$\{F\}$$

O conjunto inicial é **insatisfazível**, e aqui bastou a propagação unitária (nunca foi preciso “adivinhar” um valor). Os slides propagam alguns literais ao mesmo tempo (p_3 e $\neg p_7$; depois $p_4, p_5, \neg r$ e p); a ordem não altera o resultado.

O algoritmo DLL

Quando a propagação pára e ainda não se chegou a uma conclusão (não há cláusulas unitárias, o conjunto não é vazio e não contém F), é preciso **escolher** um literal e experimentar os dois valores.

Definição — Algoritmo de Davis-Putnam (DLL)

DLL(S)

input: conjunto de cláusulas S

output: satisfazível ou insatisfazível

S := propagate(S)

if S vazio then return satisfazível

if S contém F then return insatisfazível

l := select_literal(S)

if DLL(S uniao {l}) = satisfazível

then return satisfazível

else return DLL(S uniao {~l}) -- ~l: o complementar de l

- Acrescentar a cláusula unitária {l} é **supor** $l = V$; a propagação da chamada recursiva trata do resto.
- Se essa suposição dá insatisfazível, **volta-se atrás** (*backtracking*) e supõe-se o contrário, $\tilde{l}$.
- Só se responde “insatisfazível” quando os dois ramos falham.

A função `select_literal` devolve um literal de uma das cláusulas. Pode ver-se como um **parâmetro** do algoritmo: uma boa escolha torna-o muito mais eficiente. Critérios possíveis, escolher uma variável que:

- ocorre mais vezes;
- tem o **produto** das ocorrências de l e de $\tilde{l}$ **máximo**;
- ocorre mais vezes em **cláusulas de tamanho mínimo**;
- etc.

Atenção — Ler uma valoração da resposta

Nota adicional (não estava explícito nos slides). Quando o DLL responde “satisfazível”, os literais que foram **supostos** e **propagados** no ramo que teve sucesso formam uma valoração que satisfaz S . As variáveis que não apareceram podem ter qualquer valor.

Exemplo resolvido — Exemplo dos slides: é preciso escolher

$$S = \{\neg p \vee \neg q, \neg p \vee q, p \vee \neg q, p \vee q\}$$

Não há cláusulas unitárias: a propagação não faz nada. Escolhe-se um literal, por exemplo $\neg p$.

Ramo $S \cup \{\neg p\}$. Propagar $\neg p$: saem as cláusulas com $\neg p$ ($\neg p \vee \neg q$ e $\neg p \vee q$). Às que têm p tira-se p : $p \vee \neg q$ fica $\neg q$ e $p \vee q$ fica q . Obtém-se $\{\neg q, q\}$. Propagar $\neg q$: sai $\neg q$ e q fica vazia: $\{F\}$. **Falha.**

Ramo $S \cup \{p\}$. Propagar p : saem $p \vee \neg q$ e $p \vee q$; $\neg p \vee \neg q$ fica $\neg q$ e $\neg p \vee q$ fica q . Outra vez $\{\neg q, q\} \Rightarrow \{F\}$.

Falha.

Os dois ramos falham: o algoritmo devolve **insatisfazível**. (Faz sentido: as quatro cláusulas proíbem cada uma das quatro combinações de valores de p e q .)

Exemplo resolvido — Exercício 1.22(a) (lab, proplogic.pdf)

$$S = \{p \vee q \vee r, \neg p \vee \neg q \vee \neg r, p \vee \neg q \vee \neg r, p \vee q \vee \neg r, \neg p \vee q, \neg p \vee r, p \vee \neg q \vee r\}$$

Não há unitárias. Critério “ocorre mais vezes”: p está nas 7 cláusulas, q e r em 6. Escolhe-se p .

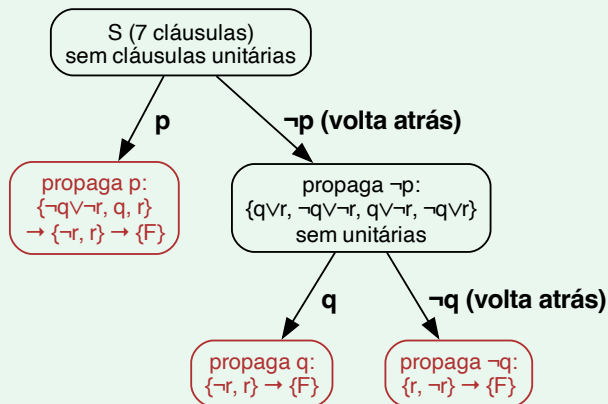
Ramo p . Saem as 4 cláusulas com p . Às 3 com $\neg p$ tira-se $\neg p$: $\{\neg q \vee \neg r, q, r\}$. Propagar q : $\neg q \vee \neg r$ fica $\neg r$, $\{\neg r, r\}$. Propagar $\neg r$: r fica vazia, $\{F\}$. **Falha.**

Ramo $\neg p$. Saem as 3 cláusulas com $\neg p$. Às 4 com p tira-se p : $\{q \vee r, \neg q \vee \neg r, q \vee \neg r, \neg q \vee r\}$. Não há unitárias; q e r ocorrem 4 vezes cada. Escolhe-se q .

• **Ramo q :** saem $q \vee r$ e $q \vee \neg r$; $\neg q \vee \neg r$ fica $\neg r$ e $\neg q \vee r$ fica r . $\{\neg r, r\} \Rightarrow \{F\}$. **Falha.**

• **Ramo $\neg q$:** saem $\neg q \vee \neg r$ e $\neg q \vee r$; $q \vee r$ fica r e $q \vee \neg r$ fica $\neg r$. $\{r, \neg r\} \Rightarrow \{F\}$. **Falha.**

Todos os ramos falham: **insatisfazível.**

**Pratica agora — Davis-Putnam (proplogic.pdf, 1.22 e 1.23)**

- **1.22 (c)** $\{p \vee \neg q \vee \neg r, \neg p \vee q \vee r, \neg p \vee q \vee r, \neg p \vee q, \neg p \vee r\}$. Repara na cláusula repetida. Depois de escolher o primeiro literal, a propagação faz o resto.
- **1.22 (e)** $\{p \vee \neg q \vee \neg r, \neg p \vee q \vee r, \neg p \vee \neg q \vee \neg r, \neg p \vee \neg q \vee r, \neg p \vee q \vee \neg r, p \vee q \vee \neg r\}$. Começa por p , como na 1.22(a); aqui é preciso voltar atrás antes de encontrar uma valoração. Diz qual é.
- **1.23** Implementar o algoritmo DLL (propagação + escolha + volta atrás), a partir do pseudo-código acima.

A 1.22(b) e a 1.22(d) são a mesma técnica com outros conjuntos; não é preciso fazê-las todas.

Soluções, uma alínea de cada vez: *SOLUCOES_Logica.html*